
SOFTMAX EXPRESSIVENESS WITH LINEAR COMPLEXITY BY INVESTIGATING THE LINEAR-SOFTMAX ATTENTION ACCURACY GAP

Gabriel Mongaras
Lyle School of Engineering
Southern Methodist University
Dallas, TX 75205
gabriel@mongaras.com

Eric C. Larson
Lyle School of Engineering
Southern Methodist University
Dallas, TX 75205
eclarson@smu.edu

May 1, 2026

ABSTRACT

Softmax attention (Vaswani et al., 2017) (Bahdanau et al., 2015) has gained traction as one of the most important parts of most modern machine learning models. Despite being the most adopted modern machine learning algorithm, exploration of softmax attention itself has been neglected. While softmax attention is a very powerful architectural component, it has a major drawback of being quadratic in sequence length. More specifically, softmax attention has quadratic complexity during training and linear complexity (when using a KV cache) during inference, which limits the sequence length it can model due to system memory constraints. Linear attention was proposed, (proposed by (Katharopoulos et al., 2020)) was created to overcome this quadratic bottleneck, having linear complexity during training and constant complexity during inference. While the complexity of linear attention is very favorable, the accuracy of naive linear attention models is not. Naive linear attention replaces the exponential in softmax with a decomposable kernel function. Popular choices of this kernel function, such as *ReLU*, lead to significantly worse modeling accuracy than softmax attention, when trained under the same model and data configuration. The first question this thesis attempts to answer is why this accuracy gap exists. Specifically, the discrepancy gap can be seen via the Taylor expansion of the numerator while the softmax denominator can be interpreted as a choice of normalization. Understanding the accuracy gap between softmax and linear attention is important to understand, however one is still left with the choice of a very accurate method that is undesirable in memory, or a method that is much better in memory while also being much worse in accuracy. Mamba2 (Dao and Gu, 2024) bridges the gap between these two methods. By relating State Space Models (SSMs) (Gu et al., 2022) to linear attention, Mamba adds a decay gate (or A-mask) to the hidden state, among other additions. The second part of this thesis uses insights from the first part to improve Mamba2 to be as accurate as softmax attention, while retaining linear train complexity. Code is provided for both sections (section 3¹) and (section 4²).

1 Background and Related Work

1.1 Transformers

Transformers (Vaswani et al., 2017) were introduced to replace Recurrent Neural Networks (RNNs) (Rumelhart et al., 1986) for sequential modeling tasks. In particular, the transformer was developed for natural language translation. The original design consisted of an encoder and a decoder (Figure 1). The encoder consists of alternating attention (Bahdanau et al., 2015) and inverse bottleneck Multilayer Perceptron (MLP) or feedforward layers. In particular,

¹<https://github.com/gmongaras/On-the-Expressiveness-of-Softmax-Attention-A-Recurrent-Neural-Network-Perspective>

²<https://github.com/gmongaras/2Mamba2Furious>

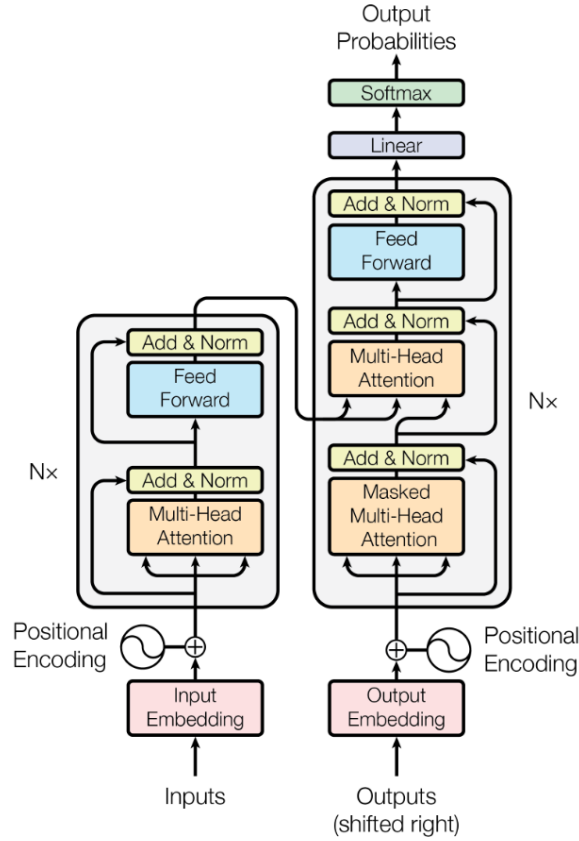


Figure 1: Transformer as shown in Attention is All You Need

the encoder uses bidirectional attention, and encodes the original sequence to translate. The decoder stacks blocks of causal attention, cross attention, and feedforward layers, similar to the encoder, to produce the translated output sequence. While traditional RNNs required the RNN block to do a forward pass for each input and output (of which are often referred to as tokens), the transformer requires a single encoder pass on the input sequence and a single decoder pass on the output sequence during training. However, to train the output on a single forward pass, causality must be preserved. To achieve a fully causal decoder, a causal mask is used on self-attention blocks in the decoder to prevent future tokens from interacting with past tokens. While the causal mask prevents information from flowing backward, it makes training large language models (LLMs) feasible. Self attention in the encoder is used to model input sequence self-interaction and is bidirectional as the entire input sequence is known at inference time. Cross attention is used to retrieve information from the encoded input when translating in the decoder and is also not masked. Feedforward layers operate on each vector and transforms each token independent from the others.

While attention was initially designed for translation, the transformer has been adopted for a wide variety of machine learning architectures and can often be found in most modern model designs such as in text modeling (Devlin et al., 2018), (Brown et al., 2020a), image processing (Dosovitskiy et al., 2021), object detection (Carion et al., 2020), time series forecasting (Zhou et al., 2021), speech recognition (Gulati et al., 2020), graph processing (Veličković et al., 2018), protein modeling (Jumper et al., 2021), multimodal architectures (Radford et al., 2021), robotics (Brohan et al., 2023), reinforcement learning (Chen et al., 2021), and generative models (Ramesh et al., 2022) (Kong et al., 2025) (Radford et al., 2023) (Peebles and Xie, 2023) to name a few. In each application, the major difference is the input data construction process and mix of encoder / decoder layers, while the base transformer model remains mostly unchanged.

1.2 Softmax Attention

Softmax attention is used to model temporal token relationships in the transformer. While MLPs have a large set of interesting improvements, mostly in the direction of mixtures of experts (MoEs) (Shazeer et al., 2017), this work focuses on the softmax attention component, primarily in causal NLP contexts.

In the multi head setting, the embedding dimension, d is split into H heads, and attention is applied independently for each head before recombining and projecting the outputs (Devlin et al., 2018; Radford et al., 2018). This split into heads can also be grouped across query heads to reduce the attention KV cache, as is done in multi-query (Shazeer, 2019) and group-query attention (Ainslie et al., 2023) where keys and values are shared among multiple query heads. Denote B as the batch size, H as the number of heads, N as the sequence length for the queries, M as the sequence length for the keys and values, d_q as per-head dimension for the queries, and d_k as the per-head dimension for the keys and values. Summations $\sum_j$ are used to denote row-wise sum reductions without explicitly indexing the row dimension (for brevity). X and Y are two, potentially different, sequences, and $M \in \mathbb{R}^{B,H,N,M}$ is an additive attention mask, typically used for causal attention. Softmax in it's most general, multi-head, batch form can, be seen in equation 1

$$\begin{aligned} Q &= XW_Q \in \mathbb{R}^{B,H,N,d_q} \quad K = YW_K \in \mathbb{R}^{B,H,N,d_k} \quad V = YW_V \in \mathbb{R}^{B,H,N,d_k} \\ O &= \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} + M \right) V = \frac{\exp \left(\frac{QK^T}{\sqrt{d_k}} + M \right) V}{\sum_j \exp \left(\frac{QK^T}{\sqrt{d_k}} + M \right)} \end{aligned} \quad (1)$$

The general form of attention has a lot of added complexity, which is unnecessary for the analysis in this thesis. The algorithms in this work assume $X = Y$, that is the query, key, and value inputs come from the same sequence. This equality allows $N = M$ and $d_q = d_k$, as such, d is used for the input dimension, d_h is used for the per-head dimension, and N is generally used for the sequence dimension. Additionally, the batch dimension is dropped as it is unnecessary for the analysis. The simplified softmax attention formulation used in this work is found in equation 2 (also without the $\frac{1}{\sqrt{d_k}}$ term for simplicity).

$$\begin{aligned} Q &= XW_Q \in \mathbb{R}^{H,N,d_h} \quad K = XW_K \in \mathbb{R}^{H,N,d_h} \quad V = XW_V \in \mathbb{R}^{H,N,d_h} \\ O &= \text{softmax} (QK^T + M) V = \frac{\exp(QK^T + M) V}{\sum_j \exp(QK^T + M)} \end{aligned} \quad (2)$$

While softmax attention is a powerful architectural component, the algorithmic complexity is $O(N^2d)$, or quadratic with respect to the sequence length during training. During inference, assuming the use of a KV cache, attention is linear in sequence length. Neither of these properties are desirable for modeling long context.

1.3 Efficient Softmax Attention Algorithms

The softmax attention mechanism has a quadratic complexity in both time and memory, as the computation of QK^T is quadratic in the sequence length N and the dimensionality d of each token. Specifically, the operation has a time complexity of $\mathcal{O}(N^2d)$ and a maximum memory usage of $\mathcal{O}(N^2)$. While this quadratic complexity is manageable for short sequences, the recent success of large language models (LLMs) (Brown et al., 2020b; Chowdhery et al., 2023; Hoffmann et al., 2022) has necessitated the processing of longer sequences for in-context learning, rendering the quadratic complexity intractable (Tay et al., 2022). Several methods have been developed to reduce the quadratic bottleneck and make computing the quantity more efficient. Most notably, Flash Attention (Dao et al., 2022), (Dao, 2023), (Shah et al., 2024), (Zadouri et al., 2026) computes softmax attention in tiles for better GPU utilization. Although Flash Attention is more efficient than computing softmax attention in native PyTorch, the FLOP count remains quadratic. Other algorithms such as paged attention (Kwon et al., 2023) and ring attention (Liu et al., 2023) make attention efficient in a distributed setting. While these algorithms are much more efficient than standard attention on a single gpu, the underlying attention mechanism is still bottlenecked by its quadratic FLOP count. Therefore, other methods exist that approximate attention for efficiency.

1.4 Sparse Softmax Attention

Sparse attention mechanisms (Child et al., 2019; Kitaev et al., 2020; Beltagy et al., 2020; Zaheer et al., 2020) aim to reduce the computational complexity by attending to only a subset of the input sequence. These methods employ techniques such as local attention, global attention, and strided attention to capture both short-range and long-range dependencies efficiently. By reducing the number of attended positions, sparse attention mechanisms can achieve subquadratic complexity, making them suitable for processing longer sequences. However, the selection of the attended positions is crucial for maintaining model accuracy, and these methods may require additional tuning and domain knowledge.

Previous approaches to sparse attention mainly focus on sparsifying the attention mask, reducing complexity often to linear. For example, sliding window attention, as done in Longformer (Beltagy et al., 2020), changes the quadratic

computation, $O(N^2)$ to a linear computation $O(WN)$ where W is the window size. While the complexity of these algorithms is favorable, the accuracy is typically significantly worse than softmax attention for most tasks as the model is blind to any relationships outside the attention window. More modern approaches such as DeepSeek Sparse Attention (DSA) (DeepSeek-AI et al., 2025) use a learned router, which does have quadratic complexity, however up to a much smaller coefficient than full attention. The router chooses what tokens pass through softmax attention, reducing the actual attention computation to $O(KN)$, where K is the top- k tokens the router chose to go through the attention computation. Approaches such as DSA are much better than most sparse attention algorithms without a router, however the actual attention mechanism is still blind to anything outside of the chosen top- k context.

1.5 Linear Attention

Linear attention algorithms (Katharopoulos et al., 2020; Choromanski et al., 2021; Peng et al., 2021) aim to reduce the complexity of softmax attention from quadratic $O(N^2)$ to linear $O(cN)$, where c is some constant. Linear attention does so by replacing the exponential nonlinearity in the softmax function with a decomposable kernel function. These algorithms are linear during training and constant during inference, making them much more desirable than softmax attention, solely based on algorithmic complexity. Another advantage of linear attention is that it can be implemented as an RNN (Rumelhart et al., 1986), which increases efficiency for long sequence inference. While linear attention has linear memory and complexity during training and constant memory complexity during inference, the accuracy is consistently worse than that of softmax attention.

Linear attention is derived by taking multi head softmax attention and replacing the softmax nonlinearity function with a kernel function $\exp(QK^T) \Rightarrow K(Q, K) = \phi(Q)\phi(K)^T$, as shown in Algorithm 1. The initial linear attention algorithm (Katharopoulos et al., 2020) chose $\phi(\cdot)$ to be $\text{elu}(\cdot) + 1$ because it is positive (which is necessary for QK , softmax-like, normalization) and they found it worked better than other functions such as $\text{ReLU}(\cdot)$. By making this substitution, linear attention can either be computed by first taking the inner product of the queries and keys (equation (24)) or by first taking the inner product of the keys and values (equation (25)). The former, like softmax attention, has quadratic complexity in the sequence length (N^2d), while the latter has quadratic complexity in the embedding dimension (Nd^2). One note is that adding the causal mask requires a special kernel to make causal linear attention computationally efficient, otherwise the training complexity in naive torch is similar to softmax attention as seen in Algorithm 1. While linear attention is desirable due to its linear complexity, naive linear attention falls short of softmax attention accuracy, as shown in Figure 12.

More recent works in NLP applications make naive linear attention more expressive. Two such prominent works are Mamba (Gu and Dao, 2024), which adds semi-separable decay components to attention, and DeltaNet (Yang et al., 2024b), which uses the delta rule (Widrow and Hoff, 1960), modeling the linear attention update rule as a form of gradient descent. The accuracy of these algorithms is significantly better than vanilla linear attention but still falls short of full softmax attention. Motivated by this finding, this work aims to build on these efficient attention alternatives and increase model accuracy closer to that of softmax attention.

1.6 Linear Kernel Attention

The first approach to improve linear attention methods took a more mathematical approach through the kernel function literature (Choromanski et al., 2021) (Gan et al., 2023) (Aksenov et al., 2024) (Xiong et al., 2021). Rather than making an arbitrary choice of ϕ , these methods typically choose ϕ to produce an inner product function that projects the inner product into a higher dimensional space or has more structure than the naive linear kernel. However, most of these kernel functions are either computationally inefficient or do not produce significantly better results than normal linear attention given sufficient context length. (Han et al., 2024a) (Deng et al., 2023a)

1.7 Cosine Attention

Linear attention looks like a perfect improvement to softmax attention on paper, but in practice it tends to be less accurate than softmax attention. To explore the problems related to linear attention, our previous work conducts an investigation by developing a new algorithm, cosine attention or cottention (arguably not the best name) (Mongaras et al., 2025). As seen in equation ((3)) This method uses the cosine similarity as the linear attention kernel function, which can be decomposed by setting ϕ to the L2Norm of the input along the embedding dimension d_h : $\phi(X) = \frac{X}{\|X\|_2}$. The attention matrix is filled with cosine similarity scores between the queries and keys, which can take on a value in the range $[-1, 1]$.

Cosine attention has a massive problem that softmax attention does not. As the sequence length grows, the query-key attention matrix row sums also grow, resulting in a max row sum of N , the sequence length. To mitigate this issue

softmax normalizes with the row sums themselves, making the row-sums add up to 1. Cosine attention cannot do the same form of normalization as row-sum normalization requires non-negative values. To fix this issue, cosine attention divides by $N^{\sigma(c)}$ where $c \in \mathbb{R}^H$ is a per-head learnable constant at each layer and σ is the sigmoid function. N effectively reduces the row-sum max to 1. The $\sigma(c)$ term is a way of relaxing the row-wise sum such that it can take on any value between 0, when $\sigma(c) = 0$, and N , when $\sigma(c) = 1$. This normalization is a lightweight method for fixing the magnitude problem and stabilizes training.

$$O = \frac{1}{N^{\sigma(c)}} \odot \frac{Q}{\|Q\|_2} \frac{K}{\|K\|_2} V = \frac{1}{N^{\sigma(c)}} \odot \frac{QK}{\|Q\|_2 \|K\|_2} V \in \mathbb{R}^{N, H, d_H} \quad (3)$$

1.8 Expressiveness of Linear Attention

Many approaches have attempted to make linear attention more expressive to match softmax attention, while still retaining linear complexity. (Choromanski et al., 2021) used linear approximations of the softmax kernel to achieve more accurate linear attention mechanisms. As linear attention can be interpreted as an RNN (Katharopoulos et al., 2020), (Peng et al., 2025) proposed receptance weighted key values (RWKV) attempting to enhance the expressivity of RNNs with a receptance vector for time mixing. While this method helped to address computational complexity of RNN training, it still relied on linear attention in its formulation. Mamba (Dao and Gu, 2024) employed state space models (Gu et al., 2022) to develop an efficient and expressive form of linear attention. (Sun et al., 2025) formulated linear attention as a step in gradient decent of a hidden view. Another approach treats modeling the hidden state as a step in gradient decent (Sun et al., 2025) which can be viewed as a type of linear attention. (Behrouz et al., 2024) proposed Titans, which built upon this gradient formulation creating different variants of the hidden view gradient descent. ATLAS (Behrouz et al., 2025) develops a new kind of recurrent models, leveraging test-time compute as in (Behrouz et al., 2024). They include a proof show that recurrent models using higher order hidden states are more expressive. (Sieber et al., 2024) makes a similar derivation as section 3, but in the context of control systems, and did not apply this derivation to a recurrent architecture. (Nauen et al., 2024) makes a simple derivation, but only explores second-order models. Although all of these methods are more expressive than linear attention, they are difficult to implement efficiently in hardware, akin to RNNs, and fall short to the accuracy of softmax attention.

Previous works have also explored alternatives to the softmax function in attention mechanisms. The Reformer model (Kitaev et al., 2020) employs locality-sensitive hashing (LSH) to approximate similarity between queries and keys, while the Performer model (Choromanski et al., 2021) approximates the softmax attention using random feature maps, reducing computational complexity. The Cosformer model (Qin et al., 2022) implements a cosine-based distance re-weighting mechanism in softmax attention and applies normalization techniques to stabilize training. However, these approaches have limitations that hinder their widespread adoption. The Reformer model’s reliance on LSH may not be suitable for all sequence lengths and introduces additional computational overhead. The Performer model has been primarily evaluated on language tasks and may require further investigation for its applicability to other domains. Similarly, the Cosformer model has been tested on a limited range of tasks and may need additional experiments to validate its effectiveness and stability across various scenarios.

Other works have also investigated the use of cosine similarity in various contexts. (Rabe and Staats, 2021) employed cosine similarity in self-supervised learning for visual representations, while (Nguyen et al., 2023) utilized it in enhancing cross-modal retrieval. (Luo et al., 2018) and (Crowson et al., 2024) explored cosine similarity in the context of convolutional neural networks and scalable attention mechanisms, respectively. However, these approaches either encountered stability issues or reverted to using the standard softmax attention, suggesting room for further improvement in leveraging cosine similarity effectively in attention mechanisms.

1.9 Mamba and SSM Models

Many approaches take a recurrent view on linear attention to improve its’ expressiveness. In particular, structured state space models (SSMs) (Gu et al., 2022, 2021; Gupta et al., 2022) have emerged as a promising alternative to attention mechanisms. SSMs model the input sequence as a continuous-time process and leverage the properties of state space representations to achieve efficient computation. By parameterizing the state transitions with structured matrices, such as diagonal or low-rank matrices, SSMs can model long-range dependencies with a linear complexity. However, the accuracy of SSMs on language tasks has been limited compared to softmax attention (Gupta et al., 2022).

The original Mamba (Gu and Dao, 2024) architecture was derived from a state space model (SSM) (Kalman, 1960) (Gu et al., 2022), which are not expressive on their own due to having time-independent queries, keys, and values (referred to as C , B , and x in the SSM literature). Mamba improves SSM expressiveness by making the query, key, and value matrices time-dependent. Mamba-2 further improves efficiency and expressiveness by introducing the associative scan

algorithm and decay mask (A mask). The resulting algorithm can be effectively formulated as linear attention with a decay mask (section 4.2 expands this assertion and isolates elements of Mamba-2 that are most expressive). From this perspective, Mamba-2 takes linear attention and makes it more expressive through clever architectural elements and parallelization via associativity.

1.10 Mamba-2

The complete Mamba-2 algorithm is found in GitHub³ and has some additions not described fully in the paper (Dao and Gu, 2024). That is, the Mamba-2 codebase builds in several inductive biases. The full Mamba model with most biases included from this codebase is written out in algorithms 2 and 3. Note that the head and sequence dimensions are arbitrarily swapped to simplify the presented algorithm, though the swaps can be inferred via the explicit shapes. To make the comparison with other algorithms easier, as noted in (Dao and Gu, 2024), we employ the Q , K , and V notation, rather than the equivalent C , B , and x notation. Additional simplifications were made such as using a single head group, however like with softmax attention, one can have head groups akin to MQA in attention (Ainslie et al., 2023).

As seen in Algorithm 2 and Algorithm 3, Mamba-2 has numerous elements beyond just a decay mask. Many of the choices seem arbitrary or underexplored and add complexity without much investigation or motivation. This work questions whether the added complexity is necessary for Mamba-2 to be more accurate than vanilla linear attention. To analyze the components that make Mamba-2 superior (and propose changes), this work breaks Mamba-2 into its components and build up a simplified algorithm, which we name Mamba-2S, from these singular components.

This algorithm is used as the basis for a major part of the later analysis. In this thesis, it is described in more detail in section 4.

1.11 Gradient-based Learning

Learning to learn at test time (Sun et al., 2025) takes a different approach to sequence modeling and proposes an internal objective that each layer learns. Each layer updates a set of online weights according to the input to that layer. In particular, each layer models a loss function such as the one in equation (5). Q , K , and V follow the typically query, key, value pattern and are optimized like normal, with respect to the outer next token prediction loss. M is called the inner model (not to get confused with the M mask in attention). Typically it is chosen to be a simple linear layer, with dimensionality (d_h, d_h) or (H, d_h, d_h) , for simplicity and easy of a fast implementation. The inner model can take on a more interesting form as long as the input and output dimensions match, such as an MLP. To keep notation consistent and simple, the equations below for this model take on the form of a matrix multiplication rather than an MLP. As opposed to being optimized by gradient descent in the typical way, the inner model is optimized by taking the gradient of the loss with respect to the inner model, M , resulting in an update as in equation (6). One can view these two updates as the outer (main) model using Q , K , and V to tell the inner model, M what to update its hidden state with. After the inner model is updated, the inner model is applied to Q_t as in equation (7). The inner operation is a normal matmul when M_t is a simple matrix or a function application when M_t is a more complicated function such as an MLP. Each new input, the inner model is updated with a step of gradient descent. If the inner model is a simple weight matrix and the inner model is viewed as a dynamic hidden state, then this form looks similar to linear attention where, instead of updating the hidden state with $K^T V$, the hidden state is updated with $\nabla_{M_{t-1}} \ell(M_{t-1}; x_t)$.

$$Q_t = W_Q x_t, K = W_K x_t, V = W_V x_t \in \mathbb{R}^{H, d_h} \quad (4)$$

$$\ell(M_{t-1}; x_t) = \|k_t M_{t-1} - v_t\| \quad (5)$$

$$M_t = M_{t-1} - \eta \nabla_{M_{t-1}} \ell(M_{t-1}; x_t) \in \mathbb{R}^{H, d_h, d_h} \quad (6)$$

$$O_t = Q_t M_{t-1} \in \mathbb{R}^{H, d_h} \quad (7)$$

The gradient can be derived to form a delta learning rule (Yang et al., 2024b), (Widrow and Hoff, 1960) as in equation (8), in which the resulting model is named DeltaNet. The learning rate, η is parameterized as $\beta_t \in (0, 1)$. The equation can also be written in the form seen in equation (9), which typically shown in other literature. This update rule can be viewed as updating the hidden state with the key-value outer product, as in normal linear attention, but also removing a key-pair from the hidden state, effectively allowing the model to both add and remove information for each recurrent update.

³<https://github.com/state-spaces/mamba>

$$M_t = M_{t-1} - \beta_t k_t^T (M_{t-1} k_t - v_t) \in \mathbb{R}^{H, d_h, d_h} \quad (8)$$

$$M_t = M_{t-1} (I - \beta_t k_t^T k_t) - \beta_t k_t^T v_t \in \mathbb{R}^{H, d_h, d_h} \quad (9)$$

The idea of test time training has been adopted by multiple other works (Behrouz et al., 2024) (Behrouz et al., 2025), which aim to improve the expressiveness of this type of model. This line of work is similar to work that extended linear attention. The initial test time training implementation can be viewed as a form of standard linear attention while additional work attempts to make it more expressive by adopting strategies typically used in optimization. For example, Titans (Behrouz et al., 2024) adds momentum to the inner function update rule.

While this line of work is very interesting and promising, it is still rather early in development. The original work is similar to that of linear attention, and while it is better than linear attention in terms of accuracy, it still falls short of softmax attention. Additionally, newer methods such as the ones proposed in Atlas (Behrouz et al., 2025) and Titans (Behrouz et al., 2024) are complicated to implement, are much less efficient than standard linear attention approaches, and have no open source implementations.

1.12 GatedDeltaNet

Among many improvements to DeltaNet, the most prominent method is Gated DeltaNet (Yang et al., 2025), which combines the strengths of both Mamba-2 (forgetting gate) and DeltaNet (key removal). Equation (10) shows the resulting equation for Gated DeltaNet, which only requires an additional $\alpha_t \in (0, 1)$ value on the previous hidden state multiplicative update term.

$$M_t = M_{t-1} (\alpha_t (I - \beta_t k_t^T k_t)) - \beta_t k_t^T v_t \quad (10)$$

While this method combines many strengths of modern linear attention architectures, it still falls short of softmax-level accuracy at scale and is difficult to implement efficiently due to the multiplicative hidden state update requiring a matrix-multiplication at each step.

1.13 Hybrid Attention Architectures

Most softmax attention literature aims to optimize softmax attention to be more efficient while most linear attention literature aims to make linear attention more expressive. Another line of work takes the strengths of both linear attention and softmax attention and scales the combination to large-scale architectures. This line of work shows that combining linear attention with softmax attention produces an architecture that is more memory efficient than normal softmax attention, yet just as accurate. Additionally, the combination of softmax attention and linear attention can theoretically model problems neither can individually. Because softmax attention stores all keys and values, it is very good at retrieval tasks, however since softmax is essentially an additive RNN, there are problems it cannot model. Linear attention is worse at retrieval tasks due to its limited sized hidden state, however more complicated linear attention mechanisms, such as Mamba-2 and DeltaNet, can model problems that additive softmax attention cannot. While the combination of the two methods is more efficient than standard softmax attention, the resulting architecture is still quadratic in memory and requires a careful balancing of the softmax-linear attention layer ratio. (Team et al., 2025), (MiniMax et al., 2025), (Merrill et al., 2026), (Chen et al., 2026), (Wang et al., 2025)

1.14 Why Softmax Attention?

There are several works that also attempt to address the question: What makes softmax attention so accurate? This fundamental question has eluded researchers for some time. For example, (Miller, 2023) noticed softmax attention cannot “zero out” attention heads. More specifically, when $\lim_{x_i \rightarrow \inf}$ for all tokens, x_i , we want the head output to produce zero, thus giving no weight to any tokens. In actuality, it produces a uniform distribution over tokens instead. To fix this issue, (Miller, 2023) adds one to the denominator. This work emphasizes how softmax has subtle problems even though it works well in practice. (Smith, 2025) asks why attention works, building upon (Miller, 2023) to change the attention mechanism. However, the focus of this work is on the improvements to attention, rather than exploring exactly what makes attention work. (Deng et al., 2023b) examines the accuracy gap between linear and softmax attention, but only in a classification and empirical context. Why linear attention is lacking compared to softmax attention, in general, remains not fully understood. (Han et al., 2024b) shows that linear attention is lacking an injective property and cannot model local features while softmax attention is injective and can model local features. (Collins et al., 2024) uses Lipschitzness to explain in-context learning in softmax. These works provide intuition but, ultimately, these properties do not fully explain the accuracy gap.

1.15 Related Work Summary

Method	Problem
Softmax attention	Quadratic in memory
Sparse softmax attention	Model is context blind
Linear attention and kernel methods	Neither accurate nor expressive
Mamba-2, Deltanet, Gated Deltanet	Still not as accurate as softmax attention
Hybrid models	Quadratic memory requirement

Table 1: Summary of related work

Table 1 summarizes the related work section. Most architectures either have a quadratic memory cost or are less accurate than softmax attention. Additionally, current work on describing why softmax attention is better than linear attention takes on a more empirical investigation. The work presented in this thesis aims to provide a concrete explanation as to why softmax attention is more accurate than linear attention, then use this explanation to form a new linear attention method that is more accurate than current methods, yet more efficient than softmax attention.

2 Motivation and Contributions

2.1 Complexity of Softmax and Linear Attention

This thesis starts the analysis of linear attention and softmax attention by first motivating linear attention as an interesting attention alternative that is worth exploring due to its linear complexity. This motivation uses concepts from related work (section 1). For readers that want a review of linear and efficient attention, we recommend first reading section 1. Reiterating from related work, linear attention is formulated by replacing the exponential on the queries and keys with a decomposable kernel function, ϕ . That is, $e^{QK^T} \approx \phi(Q)\phi(K)$. The transformation is shown between equation (11) and equation (12). Note that the mask is removed for simplicity. The argument of linear complexity still holds with the mask, however one must develop a non-trivial algorithm to actually achieve an algorithm that is linear complexity, which is one of the big improvements Mamba-2 introduces, along with the Mamba-2 method.

$$O = \frac{e^{QK^T} V}{\sum_j e^{QK^T}} \quad (11)$$

$$O = \frac{\phi(Q)\phi(K)^T V}{\sum_j \phi(Q)\phi(K)^T} \quad (12)$$

During training, one must compute the entire attention matrix. Softmax attention has quadratic complexity, $O(N^2 d_h)$ with respect to the sequence length, making it an inefficient method for processing long sequences. By replacing the exponential with a decomposable kernel function, linear attention can take on the same complexity by first doing the query-key inner product, $O(N^2 d_h)$, or linear complexity in the sequence length by first doing the key-value inner product ($N d_h^2$). For any reasonable length sequence, $d_h \ll N$. Modern LLMs have a context length up to millions of tokens while the head dimension remains small, typically taking on a value not larger than 256 for some of the larger models.

Causal models have a nice benefit during inference. When deployed with softmax attention, there are two stages for processing data with an autoregressive transformer. In this example, the language of LLMs is used, however these ideas translate to any modality that uses causal attention. The first stage is typically referred to as prefill and can be thought of as pre-processing the entire prompt given to the model. This first stage is similar to training in that it processes all tokens at the same time and is quadratic in complexity with respect to the number of tokens in the input prompt. While processing the prompt, each token at each attention layer produces a query, key, and value. Keys and values can be cached in what is known as a KV-cache because information cannot be sent backward. The second stage autoregressively generates new tokens one at a time. A naive algorithm will pass all tokens through the model for each new token generated, which is necessary for bidirectional attention. However for causal attention one can utilize the KV cache produced during prefill, and update the KV cache for each new token generated to have a much more efficient algorithm. When utilizing the KV cache in the second, decode stage of inference with causal attention models, complexity is based on the KV cache, which grows by a key and value for each new token generated. The complexity during inference is linear, $O(Nd)$. While linear inference complexity does appear to be a nice property of causal attention, this property is still limiting as the KV cache slows down inference due to attention being memory bound and puts limits on how many tokens can be generated as memory grows for each token generated.

As seen in equation (14), linear attention takes on a dual-recurrent form. To process each token, the key-value outer product produces a matrix of shape (d_h, d_h) . This outer product update is then added to the current hidden state. In

other words, causal linear attention during inference is an additive recurrent neural network, where the hidden state update is the outer product of the keys and values. After updating the hidden state, which is of shape (d_h, d_h) , the queries project the hidden state, or observe it, to get the output of shape d_h (per token and per head). A diagram of linear attention can be seen in Figure 3. Linear attention during inference also has both stages, prefill and decode. Prefill takes on the training complexity, which can either be quadratic, $O(N^2d)$ or linear $O(Nd^2)$. One can think of prefill as updating the hidden state for all tokens. For each new token generated during the decode stage, the hidden state is updated and observed, producing a new output token. As linear attention is a recurrent neural network, the decode stage is constant, $O(d^2)$, in memory with respect to the sequence length.

$$\begin{aligned} H_{1:t}^K &= [H_{1:t-1}^K; k_t] \in \mathbb{R}^{t, H, d_H} \\ H_{1:t}^V &= [H_{1:t-1}^V; v_t] \in \mathbb{R}^{t, H, d_H} \\ O_t &= \text{softmax} \left(Q_t (H_{1:t}^K)^T \right) H_{1:t}^V \in \mathbb{R}^{H, d_H} \quad (13) \end{aligned}$$

$$\begin{aligned} H_t &= H_{t-1} + k_t^T v_t \in \mathbb{R}^{H, d_H, d_H} \\ O_t &= Q_t H_t \in \mathbb{R}^{H, d_H} \quad (14) \end{aligned}$$

To compare linear attention and softmax attention during the decode stage of inference directly, the KV cache can be viewed as an RNN where the update rule is key and value concatenation (equation (13), Figure 2) while linear attention can be viewed as an RNN where the update rule is addition (equation (14), Figure 3). From this point of view, softmax attention will have linear complexity and linear attention will have constant complexity during inference. Additionally, it may be obvious that concatenation (softmax) is a strictly more powerful update rule than addition (linear) as concatenation does not compress the space the keys and values are stored in while an additive hidden state does.

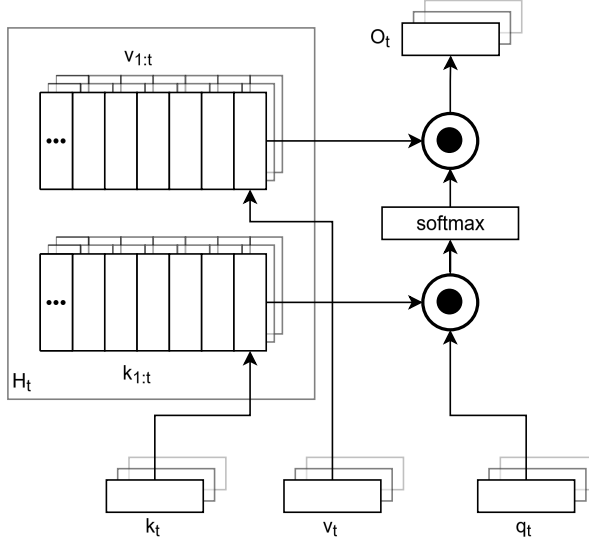


Figure 2: Softmax attention as an RNN with a KV cache. Note that the hidden state grows with each step via concatenation.

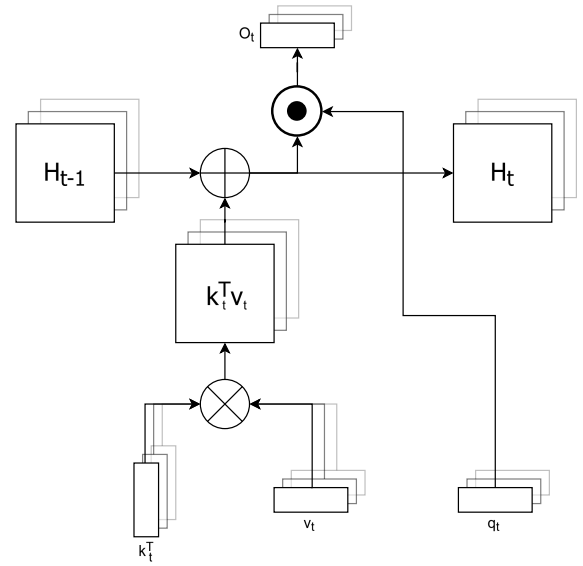


Figure 3: Linear attention as an additive RNN.

2.2 Accuracy of Linear Attention

Training this new linear attention method shows that it is very close to softmax attention in train loss. However, when doing this analysis the train loss is very noisy and a poor metric for algorithm comparison. As seen in Section 3.9, linear attention is generally less accurate than softmax attention for a variety of linear attention implementations.

Through the development of cosine attention, this analysis finds that training loss 4 looks deceptively close to softmax attention. In practice, the training loss of linear attention methods with a simple choice of kernel function are not as accurate as softmax attention. As seen with the loss curves in Section 3.9, the train loss and test loss are significantly different. Linear attention has a reasonable train loss, but the test loss is quite off.

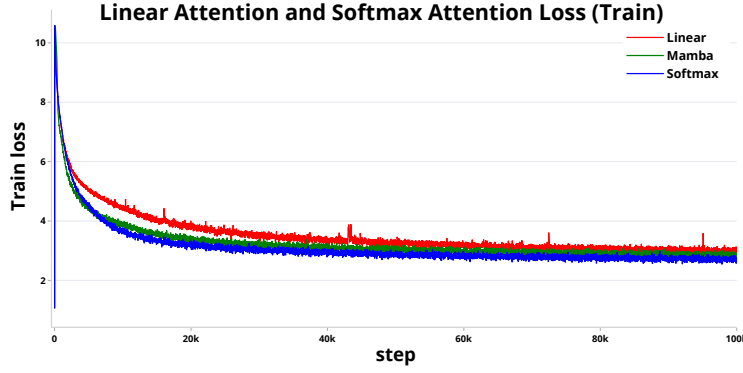


Figure 4: Loss of linear attention, Mamba, and softmax attention, keeping everything but the attention mechanism constant across experiments.

2.3 What Makes Softmax More Accurate?

It is clear that linear attention is a very efficient method, much more efficient than softmax attention. However, the accuracy is significantly worse for linear attention than that of softmax attention when trained on the same task. This observation generalizes to most applications where there is sufficient data to train a model (Deng et al., 2023b). The reason as to why linear attention is significantly less accurate than softmax attention is still a question that has not been sufficiently explored and is one that is motivated by the results from cosine attention Mongaras et al. (2025). Why does making a simple swap of activation function result in such a large accuracy degradation?

Intuition says this accuracy drop is likely because of the finite hidden state of linear attention. However, intuition is not a satisfying nor rigorous conclusion. This thesis answers this question with a formal proof that linear attention is a linear approximation of softmax attention and that softmax attention can be viewed as an infinite sum of increasing sized RNNs.

Directly looking at the softmax function, one explanation for the accuracy of softmax attention tends to describe it as a distribution, which makes softmax more accurate in some way. However, softmax is not used as a distribution in the typical sense when used in attention as it is not sampled nor does it describe probabilities of observations. Rather, softmax is used to create weights for a weighted sum. While softmax does have properties of a distribution, it is not used as a distribution and, as such, this explanation is not very helpful or meaningful. An alternative way of viewing softmax in attention is, rather than as a distribution, as being a function with nice properties. Perhaps the latent properties of the softmax function are what make softmax attention so much better than linear attention? In methods and results (section 3, these properties are analyzed in detail and can directly explain the reason linear attention is less accurate than softmax attention.

2.4 Can Linear Attention Accuracy be Improved?

Answering the "why" of the linear-softmax accuracy gap leaves an unsatisfactory result as the accuracy gap still exists. It would be desirable to have a linear method that is as accurate as softmax attention. If the properties of softmax attention are what make it so much more accurate than linear attention, can linear attention adopt some of these properties to bridge the accuracy gap? While Mamba-2 and Gate DeltaNet have attempted to bridge the gap between softmax attention and linear attention by adding a decay gate or value removal mechanism, this thesis utilizes insight from the accuracy analysis to improve linear attention with a higher order hidden state. Specifically, Mamba-2 is used as a base method for its accuracy over naive linear attention and minimal overhead during training. Mamba-2 is first simplified, as the codebase has several redundant inductive biases, then improved upon by increasing the hidden state to a second order hidden state and changing the gating method. Additionally, this thesis shows that this method is closely related to the forgetting transformer by exponentiating the query-key inner product.

2.5 Summary of Contributions

To motivate the exploration of the linear attention vs. softmax attention accuracy gap, linear attention was observed to be significantly less accurate in a variety of cases as mentioned in other works. In particular on text data, this result was confirmed. The first part of this thesis comes from "On the Expressiveness of Softmax Attention: A Recurrent Neural Network Perspective" (Mongaras and Larson, 2025), which analyzes the accuracy gap between softmax and

linear attention. The second part of this work comes from "2Mamba2Furious: Linear in Complexity, Competitive in Accuracy" (Mongaras and Larson, 2026), which develops a linear attention method as accurate as softmax attention.

3 Methods and Results - Softmax is an RNN

This section takes the linear-softmax accuracy gap and directly analyzes where it comes from by directly relating softmax to linear attention via a Taylor series expansion. Before beginning the analysis of softmax attention, notation is laid out to help avoid possible ambiguity in operations and matrix sizes.

3.1 Notation

A matrix: $A \in \mathbb{R}^{d,d}$ or a tensor: $A \in \mathbb{R}^{d,d,d}$ is denoted with a capital letter

A matrix with a "slicing index" represents a matrix sliced from the first index to the second. For example, if $t \leq T$ and $A \in \mathbb{R}^{T,d}$, then $A_{1:t} \in \mathbb{R}^{t,d} \subseteq \mathbb{R}^{T,d}$

A matrix with an index is a vector. For example if $A \in \mathbb{R}^{D,d}$ then $A_i \in \mathbb{R}^d$ where $1 \leq i \leq D$ indexes along the first axis, D .

A vector can also be denoted with an index, as done in deltanet. For example : $q_t = Q_t \in \mathbb{R}^d$. The two notations are used to keep consistency with other literature.

A vector is denoted with a lowercase letter: $a \in \mathbb{R}^d$

A vector with parenthesis and then an index is used a very small number of times and denotes a scalar: $(a)_i \in \mathbb{R}$. The parenthesis are necessary to differentiate from vectors with indices.

3.2 Softmax is an RNN Preliminaries

We first motivate our approach using the traditional formulation of causal softmax attention for calculating the output of the layer, O_t , at time t :

$$O_t = \text{Softmax}(Q_t \cdot K_{1:t}) \cdot V_{1:t} = \frac{\sum_{s=1}^t e^{Q_t \cdot K_s^T} \cdot V_s}{\sum_{n=1}^t e^{Q_t \cdot K_n^T}} \in \mathbb{R}^d \quad (15)$$

Note that for simplicity in this section, the number of heads, H is dropped and d is used in place for d_H as the analysis is only on softmax, which is independent of the input and output projections. We would like to rearrange this equation such that one can interpret the attention mechanism in a recurrent form. However, the exponentiation in softmax couples Q_t and K_s , preventing a direct regrouping. For instance, in linear attention the attention function is replaced by $\phi(Q_t) \cdot \psi(K_s)^T$. This decoupling allows causal linear attention to be reformulated into a recurrent structure, as follows:

$$O_t = \phi(Q_t) \cdot \sum_{s=1}^t \psi(K_s)^T \cdot V_s = \phi(Q_t) \cdot H_t \in \mathbb{R}^{d_H} \quad (16)$$

$$\text{where } H_t = \sum_{s=1}^t \psi(K_s)^T \cdot V_s = \psi(K_s)^T \cdot V_s + H_{t-1} \in \mathbb{R}^{d,d}$$

In order to derive a recurrent representation for softmax, we analyze the numerator and denominator of softmax attention separately. We first examine causal softmax attention without the denominator and show there exists a recurrent form. We use the Taylor series expansion of softmax to achieve this formulation (Section 3.4). Having a recurrent form, Section 3.5 shows linear attention, as in equation (16), is a first order approximation of softmax attention. Section 3.4.1 shows another motivating example—the quadratic case—for an intuition of how the recurrent form expands for higher order Taylor series approximations. In Section 3.6, the denominator is reinterpreted, using the language of RNNs with results provided in Section 3.7.

3.3 Inner Product Decomposition

Let $a, b \in \mathbb{R}^d$

Define $a \odot b \in \mathbb{R}^d$ as the Hadamard product of a and b .

Define $a \cdot b \in \mathbb{R}$ as the inner or dot product of a and b

Note that the inner product is defined as $a \cdot b = \sum_{i=1}^d (a \odot b)_i = \sum_{i=1}^d (a)_i (b)_i$

Define $a^{\otimes n} = \bigotimes_{i=1}^n a = a \otimes a \otimes \dots \otimes a \in \mathbb{R}^{d^n}$ as $n - 1$ Kronecker products

The lack of operation denotes scalar multiplication.

Using the notation mention in Section 3.4, we show the decomposed form for an inner product space to the n^{th} power is equivalent to the inner product of n^{th} order Kronecker products of each vector.

$$\begin{aligned}
 (a \cdot b)^n &= \left[\sum_{i=1}^d (a \odot b)_i \right]^n && \text{By equation (2)} \\
 &= \left[\sum_{i=1}^d (a \odot b)_i \right] \left[\sum_{i=1}^d (a \odot b)_i \right] \dots \left[\sum_{i=1}^d (a \odot b)_i \right] && \text{Power to product of n terms} \\
 &= \sum_{i=1}^d \sum_{j=1}^d \dots \sum_{k=1}^d \underbrace{(a \odot b)_i (a \odot b)_j \dots (a \odot b)_k}_{\text{combinatorial multiplication from } i \dots k} && \text{By rearranging the sums} \\
 &= \sum_{i=1}^{d^n} \left[(a \odot b)^{\otimes n} \right]_i && \text{By equation (3)} \\
 &= \sum_{i=1}^{d^n} \left[(a^{\otimes n}) \odot (b^{\otimes n}) \right]_i && \text{Rearrange the nth order Kronker Product} \quad (17) \\
 &= (a^{\otimes n}) \cdot (b^{\otimes n}) && \text{Change to inner product by equation (2)} \quad (18)
 \end{aligned}$$

3.4 Recurrent Softmax Attention

Causal softmax attention (15) is composed of exponentials of inner products between the query vector, Q_t , and key vectors, K_s . Using a decomposed inner product form, causal softmax attention can be rewritten as an RNN by taking the Taylor series expansion of the exponential function. Here, we set the multiplicative inverse of the denominator to $(g)_t$ for simplicity. Although the derivation is for the causal case, it can be extended to the bidirectional case as in Section 3.4.2.

Note that, in the explanation below, we make use of several properties:

1. **Decomposed Inner Product Property:** $(a \cdot b)^n$ is equivalent to $(a^{\otimes n}) \cdot (b^{\otimes n})$ by equation (17). A full derivation of this property is available in Appendix 3.3.
2. **Inner Product Equivalence:** $a \cdot b = \sum_{i=1}^d (a \odot b)_i = \sum_{i=1}^d (a)_i (b)_i$
3. **Kronecker Products Shorthand:** $a^{\otimes n} = \bigotimes_{i=1}^n a = a \otimes a \otimes \dots \otimes a \in \mathbb{R}^{d^n}$

where $a, b \in \mathbb{R}^d$, $a \odot b \in \mathbb{R}^d$ is the Hadamard product, $a \cdot b \in \mathbb{R}$ is the inner or dot product, and the lack of an explicit operation denotes scalar multiplication. With these properties, softmax attention can be reformulated as follows:

$$\begin{aligned}
 O_t &= (g)_t \sum_{s=1}^t e^{Q_t \cdot K_s^T} \cdot V_s \\
 &= (g)_t \sum_{s=1}^t \sum_{n=0}^{\infty} \frac{1}{n!} (Q_t \cdot K_s^T)^n \cdot V_s && \text{By definition of the Taylor Series of } e \\
 &= (g)_t \sum_{s=1}^t \sum_{n=0}^{\infty} \frac{1}{n!} \sum_{i=1}^{d^n} (Q_t^{\otimes n})_i ((K_s^{\otimes n})^T)_i V_s && \text{By equation (17)} \\
 &= (g)_t \sum_{n=0}^{\infty} \frac{1}{n!} \sum_{i=1}^{d^n} \sum_{s=1}^t (Q_t^{\otimes n})_i ((K_s^{\otimes n})^T)_i V_s && \text{By rearranging sums} \\
 &= (g)_t \sum_{n=0}^{\infty} \frac{1}{n!} \sum_{i=1}^{d^n} (Q_t^{\otimes n})_i \sum_{s=1}^t ((K_s^{\otimes n})^T)_i V_s && \text{By factoring out } Q \\
 &= (g)_t \sum_{n=0}^{\infty} \frac{1}{n!} (Q_t^{\otimes n}) \cdot \sum_{s=1}^t ((K_s^{\otimes n})^T) \cdot V_s && \text{By inner product equivalence} \\
 &= (g)_t \sum_{n=0}^{\infty} \frac{1}{n!} (Q_t^{\otimes n}) \cdot H_t^n, \quad H_t^n = \sum_{s=1}^t ((K_s^{\otimes n})^T) \cdot V_s && \text{Define hidden state}
 \end{aligned}$$

Where $Q \in \mathbb{R}^{N,d}$, $K \in \mathbb{R}^{M,d}$, $V \in \mathbb{R}^{M,e}$, $G \in \mathbb{R}^N$, $Q_t \in \mathbb{R}^d$, $K_s \in \mathbb{R}^d$, $V_s \in \mathbb{R}^e$, $(g)_t \in \mathbb{R}$. N and M are the sequence dimensions, indexed by t and s respectively. d and e are the embedding dimensions where d is indexed by i . n is the n^{th} order term in the Taylor expansion.

Thus, the softmax attention numerator does have a recurrent formulation. This formulation can also be seen visually in Figure 5. Rather than the output coming from a single recurrent equation, softmax is a sum of infinite recurrent outputs, each weighted by $\frac{1}{n!}$. Each of the RNNs that comprise the output of softmax attention have a hidden state of shape $\mathbb{R}^{d^n,e}$ due to the n^{th} order Kronecker product on the queries and keys. The n^{th} order Kronecker product can be thought of as creating n^{th} order multiplicative interactions between dimensions of the keys and queries. The hidden state for each of the infinite RNNs can be thought of as accumulating information of n^{th} order interaction terms on the dimension of Q and K .

3.4.1 Quadratic Derivation

To provide a simple example of the recurrent form derived in Section 3.4, we note that $a^{\otimes 2} = a \otimes a \in \mathbb{R}^{d^2}$ and show the recurrent form for the $n = 2$ case.

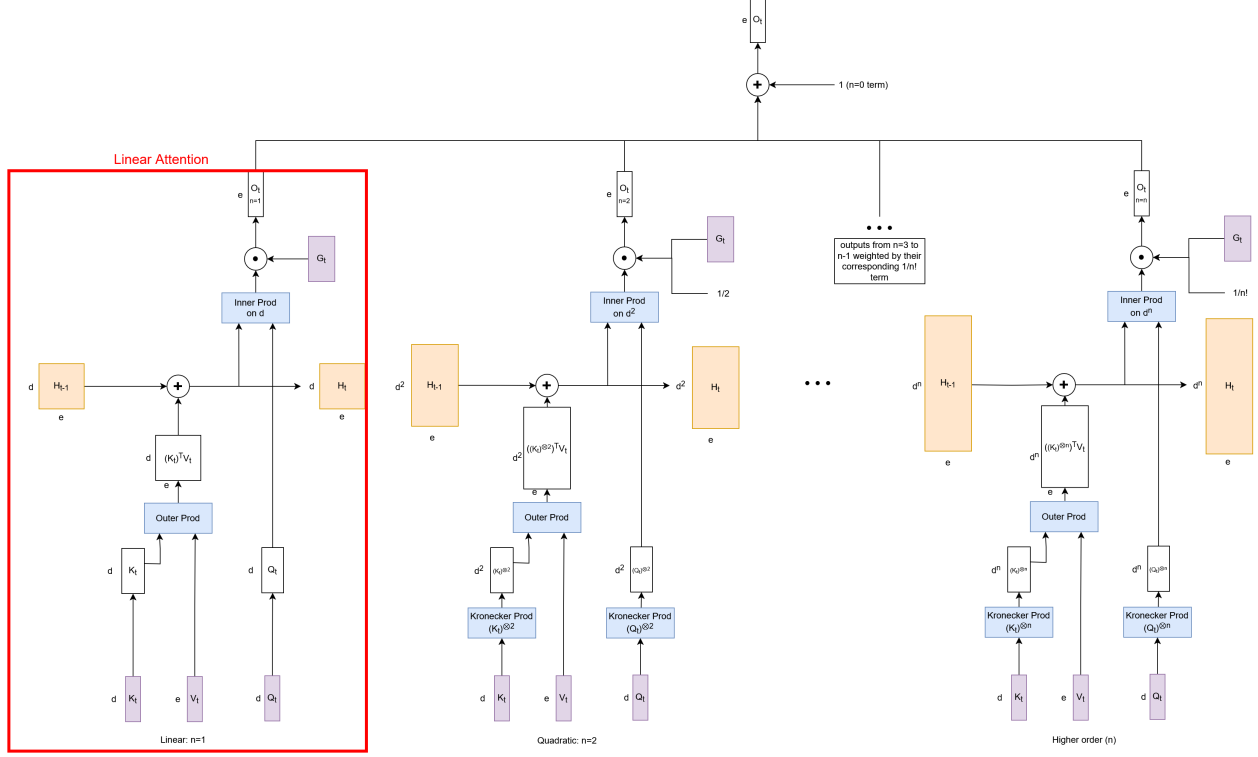


Figure 5: Softmax attention as an RNN. We define $(g)_t$ in place for the softmax denominator. Linear attention is equivalent to the $n = 1$, first order, term.

$$\begin{aligned}
 O_t &= \sum_{s=1}^t (Q_t \cdot K_s^T)^2 \cdot V_s \\
 &= \sum_{s=1}^t \left[\sum_{i=1}^d (Q_t \odot K_s)_i \right]^2 V_s && \text{By equation (2)} \\
 &= \sum_{s=1}^t \sum_{i=1}^{d^2} (Q_t \otimes Q_t)_i (K_s \otimes K_s)_i V_s && \text{By equation (17)} \\
 &= \sum_{i=1}^{d^2} \sum_{s=1}^t (Q_t \otimes Q_t)_i (K_s \otimes K_s)_i V_s && \text{rearrange summations} \\
 &= \sum_{i=1}^{d^2} (Q_t \otimes Q_t)_i \sum_{s=1}^t (K_s \otimes K_s)_i V_s && \text{By factoring out } Q \\
 &= (Q_t \otimes Q_t) \cdot \sum_{s=1}^t ((K_s^T) \otimes (K_s^T)) \cdot V_s && \text{By equation (2)} \\
 &= (Q_t \otimes Q_t) \cdot H_t \quad H_t = \sum_{s=1}^t ((K_s^T) \otimes (K_s^T)) \cdot V_s \in \mathbb{R}^{d^2, d} && \text{Define hidden state}
 \end{aligned}$$

3.4.2 Bidirectional Derivation

$$\begin{aligned}
 O_t &= (g)_t \sum_{s=1}^N e^{Q_t \cdot K_s^T} \cdot V_s \\
 &= (g)_t \sum_{s=1}^N \sum_{n=0}^{\infty} \frac{1}{n!} (Q_t \cdot K_s^T)^n \cdot V_s && \text{By definition of the Taylor Series of } e \\
 &= (g)_t \sum_{s=1}^N \sum_{n=0}^{\infty} \frac{1}{n!} \sum_{i=1}^{d^n} (Q_t^{\otimes n})_i ((K_s^{\otimes n})^T)_i V_s && \text{By equation (17)} \\
 &= (g)_t \sum_{n=0}^{\infty} \frac{1}{n!} \sum_{i=1}^{d^n} \sum_{s=1}^N (Q_t^{\otimes n})_i ((K_s^{\otimes n})^T)_i V_s && \text{By rearranging sums} \\
 &= (g)_t \sum_{n=0}^{\infty} \frac{1}{n!} \sum_{i=1}^{d^n} (Q_t^{\otimes n})_i \sum_{s=1}^N ((K_s^{\otimes n})^T)_i V_s && \text{By factoring out } Q \\
 &= (g)_t \sum_{n=0}^{\infty} \frac{1}{n!} (Q_t^{\otimes n}) \cdot \sum_{s=1}^N ((K_s^{\otimes n})^T) \cdot V_s && \text{By equation (2)} \\
 &= (g)_t \sum_{n=0}^{\infty} \frac{1}{n!} (Q_t^{\otimes n}) \cdot H_t^n, \quad H_t^n = \sum_{s=1}^N ((K_s^{\otimes n})^T) \cdot V_s \in \mathbb{R}^{d^n, e} && \text{Define hidden state}
 \end{aligned}$$

The only difference between the causal and bidirectional formulations is the range on the sequence sum indexed by s . In the causal case, the index ends at t while the bidirectional case sums to the end of the sequence. This means the hidden state operates on the entire sequence for each token rather than just the past $s \leq t$.

3.5 Linear Attention is a First Order Approximation

With this recurrent formulation, notice that taking the $n = 1$ term of the Taylor Series sum results in the well known form of linear attention (equation 16).

$$O_t^{(linear)} = \frac{1}{1!} (Q_t^{\otimes 1}) \cdot \sum_{s=1}^t ((K_s^{\otimes 1})^T) \cdot V_s = Q_t \cdot \sum_{s=1}^t K_s^T \cdot V_s = Q_t \cdot H_t \quad H_t = \sum_{s=1}^t K_s^T \cdot V_s \quad (19)$$

Equation 19 and the boxed segment of Figure 5 show that linear attention is a linear approximation of softmax attention when $\phi = \psi = I_d$. As linear attention is just a single term in softmax attention, this derivation shows how linear attention is a subset of softmax attention and provides an intuitive explanation as to why linear attention is typically less accurate. Even when other functions are used for ϕ and ψ , they can only manipulate a single recurrent chain—they cannot model the higher order terms from softmax. These terms in softmax attention allow it to model combinatorial interactions between inner product dimensions—thus, it is at least as expressive as linear attention. Also notice that each subsequent RNN comprising softmax uses a larger hidden state, modeling higher-order interactions on the combinatorial dimension of Q and K . Linear attention works with a single, smaller hidden state operating only on the dimension of Q and K , without higher-order interactions. To highlight the difference between softmax attention and linear attention, we also derive a quadratic approximation in Section 3.4.1, which models pairwise interactions on the dimension and has a hidden state quadratically larger than linear attention.

This result raises an important difference between linear attention, with functions on Q and K , versus nonlinear attention, which uses a non-decomposable function on the inner product of Q and K . Despite the two forms appearing similar, they are fundamentally different in their ability to model higher order interactions. A function on each of the vectors, $\phi(Q_t)$ and $\psi(K_s)$ restricts the vector space dimensions of Q_t and K_s . For example, *ReLU* restricts vectors to the positive portion of the vector space. On the other hand, the exponential function on the inner product space does not restrict the vector space. Instead, this function creates n^{th} degree multiplicative factors between dimensions of Q_t and K_s for all $n \in [0, \infty)$. Practically, these multiplicative dimensions become less influential for larger n , as they are weighted by $\frac{1}{n!}$, but they cannot be disregarded entirely.

3.6 Reinterpreting the Denominator

In traditional softmax attention the denominator is calculated as:

$$(g)_t = \frac{1}{\sum_{s=1}^t e^{Q_t \cdot K_s^T}} \in \mathbb{R} \quad (20)$$

where $(g)_t$ can have values from 0 to 1⁴. Typically, these values are calculated from the same Q_t and K_s values as the numerator. However, we hypothesize that the exact normalization is not the most crucial aspect of $(g)_t$. Rather, we observe that $(g)_t$ could be interpreted like a gate or norm that stabilizes the numerator, especially for long contexts when t becomes large. This stabilization aspect is hypothesized to be the crucial function of $(g)_t$, rather than the exact form of $(g)_t$. Therefore, we reformulate $(g)_t$ as a gate via:

$$(Gate) \quad \frac{(g)_t}{t} \sum_{s=1}^t e^{Q_t \cdot K_s^T} \cdot V_s = \sum_{n=0}^{\infty} \frac{1}{n!} (Q_t^{\otimes n}) \cdot H_t^n, \quad H_t^n = \frac{(g)_t}{t} \sum_{s=1}^t ((K_s^{\otimes n})^T) \cdot V_s \quad (21)$$

This assumption of the role of $(g)_t$ allows us to interpret it like an output gate in a recurrent structure at time t . This representation also ties the softmax attention mechanisms to traditional properties of expressive recurrent networks, such as the LSTM (Hochreiter and Schmidhuber, 1997) and GRU (Cho et al., 2014). However, this interpretation as a gate may be too broad of an assumption as it also allows the sequence to grow without bound. This issue can be mitigated by dividing by the sequence length, clamping the inner product (for example, we clamp the pre-exponential value to 5), and clipping the gradient. While these tricks help ensure the gate is numerically stable, they are not particularly elegant solutions and complicate the overall implementation. Alternatively, $(g)_t$ can be interpreted as a norm at time t , normalizing the numerator according to its length and values. This can be realized (equation (22)) where $\|\cdot\|$ denotes a vector norm. The optimal type of norm is not immediately clear, and could be any number of formulations such as L_2 , RMS , or others. We test each interpretation of $(g)_t$ under a recurrent perspective, as either a gate or a norm, to understand its equivalence to the traditional softmax denominator.

$$(Norm) \quad \left\| \sum_{s=1}^t e^{Q_t \cdot K_s^T} \cdot V_s \right\| = \left\| \sum_{n=0}^{\infty} \frac{1}{n!} (Q_t^{\otimes n}) \cdot H_t^n \right\|, \quad H_t^n = \sum_{s=1}^t ((K_s^{\otimes n})^T) \cdot V_s \quad (22)$$

3.6.1 Additional Gate Information

The input and output gates referenced in this paper are very similar to that of an LSTM (Hochreiter and Schmidhuber, 1997) and similar to that used by (Yang et al., 2024a). The input gate controls how much information is being added to the LSTM memory while the output gate modulates the output of the LSTM cell. As seen in Figure 6, these gating mechanisms can be translated to the original linear attention formulation where the input gate is a scalar between $[0, 1]$ at time t , modulating the $K^T V$ outer product while the output gate is a scalar between $[0, 1]$ at time t modulating output after the $Q_t H_t$ inner product. These ideas extend to softmax attention it can be computed as an infinite sum of these RNNs, although the implementation is intractable. However, the gates can be translated to multiplicative values on the $Q K^T$ attention matrix. As the rows/queries define the output at time t , the output gate can be defined along the rows. Similarly, as the columns define the input at time s , the input gate can be defined along the columns. This idea is equivalent to applying the output gate after performing the full attention operation and applying the input gate to the values. Mathematically, this can be expressed as follows:

$$\begin{aligned} O_t &= \sum_{s=0}^t \left[(g^{in})_t e^{Q_t \cdot K_s^T} (g^{out})_s \right] \cdot V_s = (g^{in})_t \sum_{s=0}^t e^{Q_t \cdot K_s^T} \cdot [(g^{out})_s V_s] \\ O &= \left[g^{in} \odot e^{Q \cdot K^T} \odot M \odot g^{out} \right] \cdot V = g^{in} \odot \left[e^{Q \cdot K^T} \odot M \right] \cdot [g^{out} \odot V] \end{aligned}$$

Where $Q \in \mathbb{R}^{N,d}$, $K \in \mathbb{R}^{M,d}$, $V \in \mathbb{R}^{M,e}$, $g^{in} \in [0, 1]^N$, $g^{out} \in [0, 1]^M$

And $Q_t \in \mathbb{R}^d$, $K_s \in \mathbb{R}^d$, $V_s \in \mathbb{R}^e$, $(g^{in})_t \in [0, 1]$, $(g^{out})_s \in [0, 1]$

⁴ $(g)_t$ can be greater than 1 if the denominator is less than 1, which is rare, only occurring for small t .

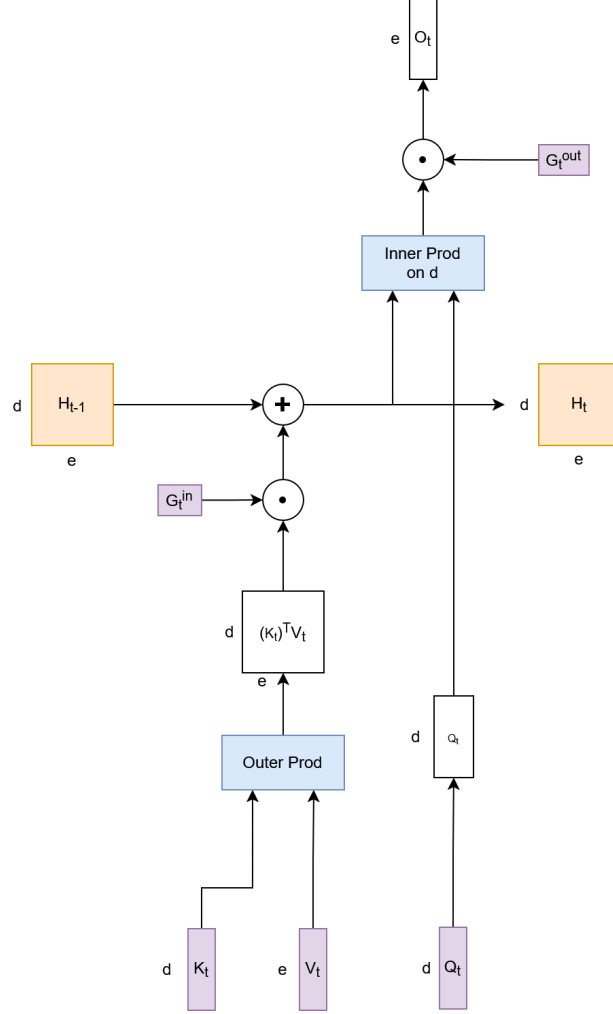


Figure 6: Linear attention as an RNN with an input and output gate.

3.7 Softmax Equivalence

To evaluate our hypothetical softmax alternative, we train multiple Llama 2 (Touvron et al., 2023) (Touvron et al., 2023) models for next token language modeling. Keeping the rest of the architecture constant, we replace the attention mechanism with the proposed variations. We show log loss to emphasize the differences between each model. Section 3.7 shows our proposed replacement is empirically equivalent to softmax. Section 3.8 examines the scalability of the proposed replacement. Section 3.9 evaluates linear attention against normal softmax and our proposed methods. As our method uses a Taylor expansion, Section 3.10 looks at the accuracy of different linear attention variations via progressive additions of higher order powers. Section 3.11 ablates various elements of the recurrent softmax attention.

Experimental Setup In our experiments, we test both replacing the denominator with a gate as in equation 21 and with a norm as in equation 22 alongside normal softmax attention. To evaluate the applicability to various domains, we retrain on three datasets: The Pile (Gao et al., 2021), SlimPajama (Shen et al., 2023), and FineWeb (Penedo et al., 2024). The Pile is a dataset created by Eleuther AI composed of 825 GiB of English text on various domains such as code, technical papers, math, and articles. SlimPajama is a 627 billion token dataset that is a cleaner subset of RedPajama (Weber et al., 2024), a dataset of various web crawl data. FineWeb is a cleaned and de-duplicated 5-trillion token dataset compiled from 96 different common crawl snapshots. Each tested model is about 300 million parameters and trained on a sequence length of 1024. Adding a gate or norm requires up to an additional d parameters per layer, which is insignificant relative to the model size. More hyperparameters for our models can be found in Appendix A.

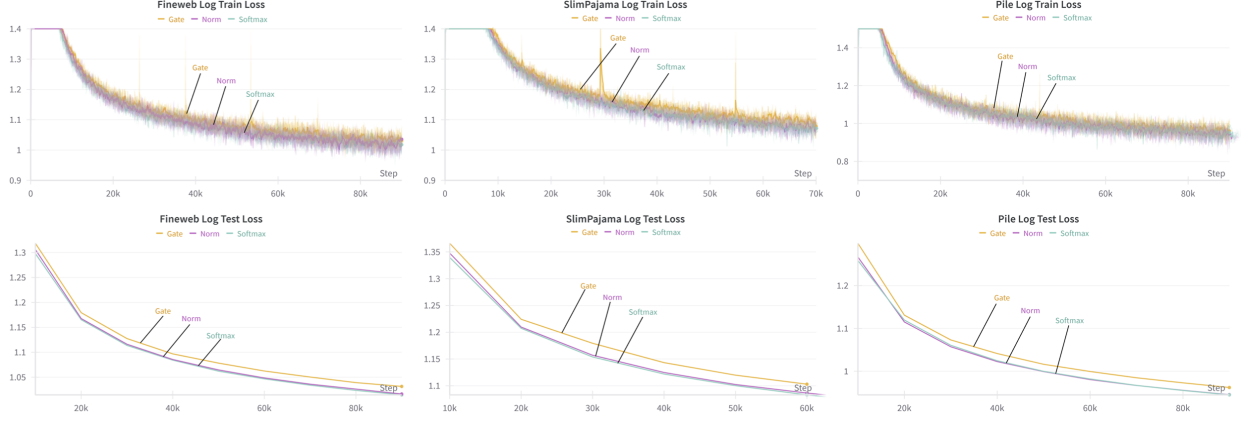


Figure 7: Test and train loss on various datasets for softmax attention and the proposed methods with gate or norm replacements. Expanded plots can be found in Appendix B.

Result Figure 7 shows the resulting loss curves for each dataset and model variant. The loss for the normed model variant follows the model trained with native softmax attention precisely while the model with a gate is slightly less accurate. We employ the L_2 vector norm for this analysis and find that it is numerically stable. The gate was semi-unstable during training. We noticed recoverable loss spikes during training where the model loss would spike and resume after some number of steps. Dividing by the sequence length, clipping the inner product before exponentiation, and performing gradient clipping, helped produce fewer spikes, however some instability was still observed. This result implies that the function of $(g)_t$ is most similar to a norm operation. Moreover, the norm does not need to mirror the traditional softmax exponentiation—a simple vector norm can suffice.

3.8 Scaling

To investigate if scaling laws hold, we scale the model in two ways. We scale the model from 300M parameters to 2B parameters, keeping all other hyperparameters constant to evaluate the scalability of the model itself. We also scale the sequence length from 1024 to 4096, keeping all other hyperparameters constant to examine how our method scales with longer sequences. These scaling are tested upon the FineWeb (Penedo et al., 2024) dataset. The scaled comparisons are found in Figure 8, showing our proposed attention formulation scales identically to softmax (with both the sequence length and the model size). Again we observe that the norm better mirrors softmax accuracy and is more numerically stable.

3.9 Linear Attention

As the proposed softmax decomposition has a direct relationship to linear attention, we evaluate the method against several variations of linear attention. Figure 9 shows that softmax attention and our proposed methods are more accurate than each linear attention variant by a significant margin. This behavior supports the assertions in Section 3.5, showing linear attention is a subset of softmax attention.

3.10 Taylor Series Terms

As our method uses a Taylor series, we investigate how model loss behaves as higher order terms are added to the less expressive attention variations. That is, we use the recurrent formulation for a particular type of linear attention and gradually add in higher order terms from the softmax Taylor expansion of softmax. The results are shown in Figure 10 for three types of linear attention: cosine similarity (Mongaras et al., 2025), ReLU (Xie et al., 2025), and $\text{elu}(x) + 1$ kernel (Katharopoulos et al., 2020). As more terms are added to the softmax approximation, the loss smoothly transitions from “linear attention loss” to “softmax attention loss.” After adding in terms up to $n = 10$, we observe that the recurrent approximation mirrors softmax with negligible differences. For the linear attention variants, however, we find that adding more terms improves accuracy but never fully reaches that of softmax attention (even for $n = 10$). We hypothesize this accuracy gap exists because linear attention variants have functions on the independent vectors, $\phi(Q)$ and $\psi(K)$, restricting the reachable vector space when combined; whereas softmax does not restrict this vector space. We leave exploring this observation to future work. We note that cosine attention does not gain any benefit from additional terms and hypothesize that due to inner product values being between 0 and 1, the resulting

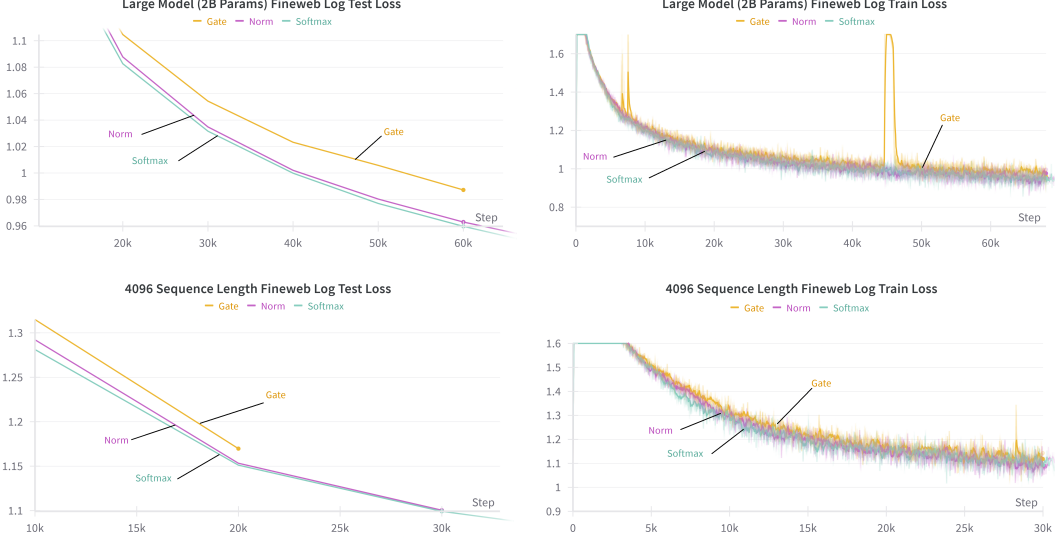


Figure 8: Test and train loss on the FineWeb dataset for large models (about 2B parameters) on 1024 sequence length and small models (about 300 million parameters) on 4096 sequence length for softmax attention and the proposed methods with gate or norm replacements. Some experiments were cut short due to time constraints

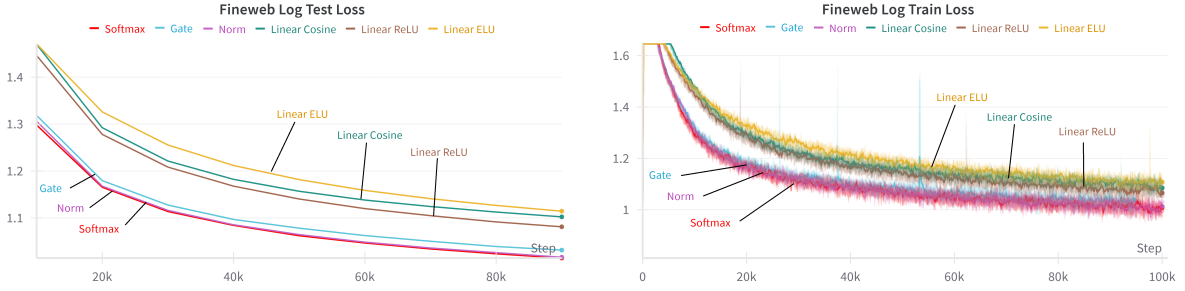


Figure 9: Test and train loss on the FineWeb dataset for various linear attention methods, softmax attention, and the proposed methods with gate or norm replacements.

higher order interaction terms are less than 1, limiting the magnitude of higher order terms and therefore the impact of these higher order terms on the resulting output.

3.10.1 Higher Order Linear Attention Expansions

For all the higher order linear attention variants the activation functions, ϕ and ψ , are applied to Q and K as in the native linear attention. The experiments above use equation (23) up to n terms starting from 0. Softmax attention can be obtained by choosing $\phi = \psi = I$ and taking terms out to $n = \infty$.

$$\begin{aligned}
 (\text{Softmax}) \quad O_t &= \sum_{n=0}^{\infty} \frac{1}{n!} (Q_t^{\otimes n}) \cdot \sum_{s=1}^t ((K_s^{\otimes n})^T) \cdot V_s &= \sum_{s=1}^t \sum_{n=0}^{\infty} \frac{1}{n!} (Q_t \cdot K_s^T)^n \cdot V_s \\
 (\text{Linear}) \quad O_t &= \sum_{n=0}^{\infty} \frac{1}{n!} (\phi(Q)_t^{\otimes n}) \cdot \sum_{s=1}^t ((\psi(K)_s^{\otimes n})^T) \cdot V_s &= \sum_{s=1}^t \sum_{n=0}^{\infty} \frac{1}{n!} (\phi(Q)_t \cdot \psi(K)_s^T)^n \cdot V_s \quad (23)
 \end{aligned}$$

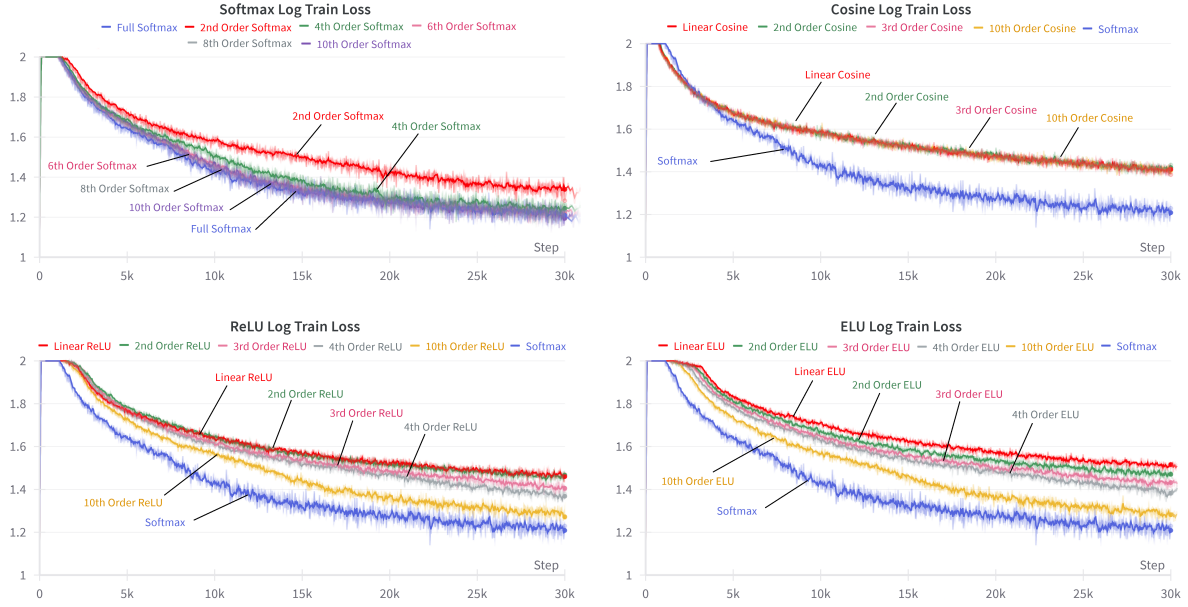


Figure 10: Log train loss for softmax attention and various linear attention method when summing more powers of the inner product. The nth order denotes the sum of powers from 0 to n.

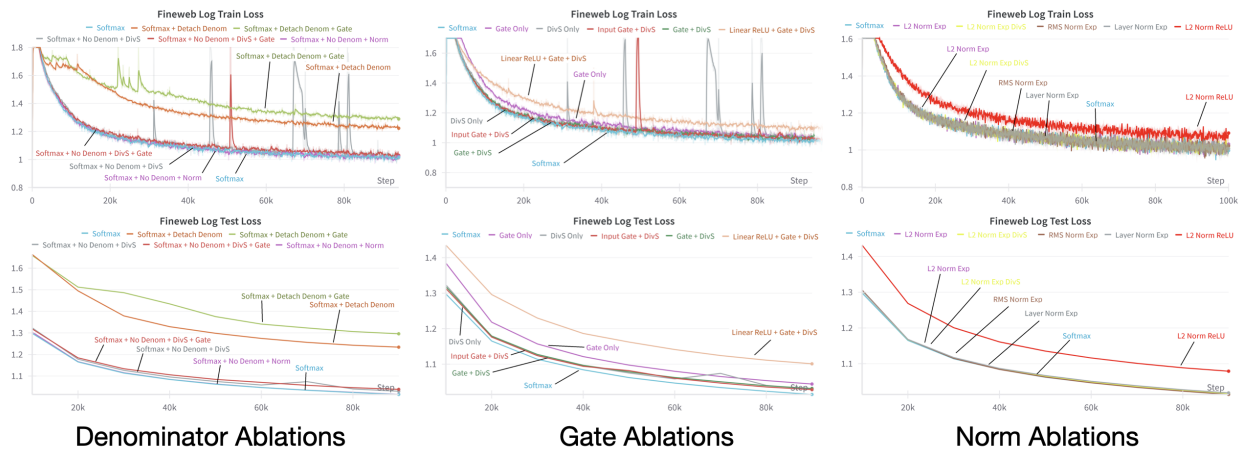


Figure 11: Test and train loss on various datasets for softmax attention and the proposed methods with gate or norm replacements. Expanded plots can be found in Appendix B.

3.11 Ablation Analysis

To further investigate various recurrent elements, we conduct an ablation study using the 300M parameter model, varying both the gated and normed variants of our method. The results of these ablations are plotted in Figure 11. The leftmost plots ablate several elements of the recurrent softmax denominator, $(g)_t$, by removing, detaching, and dividing by the sequence length, t (with varying combinations of all three). We find that detaching the denominator from the computation graph significantly hurts downstream model accuracy. Removing the denominator and dividing by the sequence length gives similar model accuracy to softmax, but also creates instability. Adding a gate helps to somewhat stabilize the training, though not entirely. Finally, we find that removing the denominator and adding a norm is what appears to mirror softmax most closely.

Gate The middle column of plots in Figure 11 ablates the gate method employed by combining sequence length normalization with an input gate or output gate (more fully explained in Section 3.6.1), and investigating ReLU linear attention with a gate. The results shows that having a gate helps to stabilize training loss, but must be accompanied by sequence length normalization for good model test accuracy. Furthermore, replacing the exponential with a decomposable ReLU kernel significantly hurts model accuracy, regardless of gating or sequence length normalization. This result emphasizes the importance of the exponential function on the inner product of Q and K , as well as indicating that some sort of sequence length normalization is required.

Norm The rightmost column in Figure 11 varies the norm method employed, investigating the L_2 norm, RMS norm, layer norm, and L_2 norm with sequence normalization. We also investigate the use of an L_2 norm with the decomposable ReLU kernel. The results shows that any type of normalization, with or without learnable parameters, works just as well as softmax. However, replacing the exponential with a decomposable ReLU kernel significantly degrades model accuracy. This graph further emphasizes that the important aspects of softmax attention are an exponential plus a vector norm. The choice of norm does not appear to influence training stability or test accuracy. However, there are differences between a vector norm (e.g., L_2) and a sequence length normalization (e.g., division by t or gating). The vector norm appears to achieve the most accurate model, implying it is a necessary aspect of softmax, but the exact form is unimportant.

3.12 Higher order attention

Before explaining the changes to Mamba-2, it is necessary to motivate our rationale through prior work. (Mongaras and Larson, 2025) examine the accuracy discrepancy between linear attention and softmax attention and found that taking the sum of higher powers of higher order RNNs approaches the accuracy of full softmax attention. By the Taylor expansion of the exponential⁵, softmax attention is equivalent to the sum of all nonnegative integer powers of the query-key inner product with the denominator being just a form of normalization. Taking the sum term-by-term, higher order terms produce a larger hidden state and improve downstream accuracy, approaching softmax accuracy.

While increasing the order of linear attention increases the expressiveness of the hidden state, it also increases the hidden state on the order of d_h^p where p is the order of the RNN and d_h is the head dimension. Vanilla linear attention turns out to be a first order approximation of softmax attention. That is, linear attention uses an additive RNN of order $p = 1$ which results in a hidden state of size (d_h, d_h) . A higher order RNN would have a hidden state of roughly dimension (d_h^p, d_h) , which can be reduced as explained in section 4.5.

⁵We note that the Taylor expansion used is more appropriately called the Maclaurin expansion, as the expansion point is centered on $x = 0$. However we adopt the same naming convention.

Purely from a memory perspective, the hidden state of softmax attention is the KV cache, which requires $2 \times N \times H \times d_h$ elements per head. The KV cache grows linearly with respect to the sequence length. While most values of p would result in an unreasonable hidden state size, a second order RNN, with $p = 2$ would result in a hidden state of dimension (d_h^2, d_h) , still reasonable in memory for a sufficiently long context window. We use this intuition to improve Mamba-2 with a second order hidden state, improving the accuracy while keeping the memory requirements lower than softmax attention for a sufficiently large context window.

Additionally, normalization on the query-key sum requires a strictly non-negative image of the query-key inner product. A second order hidden state has an image with strictly non-negative values. Linear attention obtains positive values by applying a nonlinearity on the pre-image of the inner product of the queries and keys, thus restricting the domain of the inner product function. Squaring the query-key inner product presents no such restriction on the pre-image, while keeping the inner product image positive. As such, normalization on the queries and keys rather than on the attention output can be used, which has been shown to be both efficient and stable when using a form of online softmax attention (Milakov and Gimelshein, 2018) such as flash attention (Dao et al., 2022).

3.13 Softmax as an RNN Conclusions

This section connects linear and softmax attention under a unifying recurrent formulation. A Taylor series expansion of the softmax attention numerator was employed to develop a recurrent form and competing hypotheses for approximating the softmax denominator were investigated. Linear attention was shown to be a first order approximation of softmax. Using the recurrent softmax attention formulation, equivalence was shown empirically and the crucial elements of softmax attention were analyzed in an ablation study. Additional experiments showed that a tenth order Taylor series approximation is sufficient similar to softmax. Finally, different types of normalization and gates were explored showing that any vector norm is sufficient to approximate the functionality of the softmax attention denominator. The theory developed in this work is crucial to understanding the accuracy bounds of softmax compared to other forms of attention. Moreover, this theory may be employed to uncover more accurate or efficient attention mechanisms.

Limitations The current formulation covers only linear attention and softmax attention, future work can expand it to more complicated recurrent architectures such as RWKV (Peng et al., 2025) and state-space models like Mamba (Gu and Dao, 2024). As recurrent architectures similar to RWKV and Mamba are extensions of linear attention, the theory developed in this work should extend to the additions made in these works. Only the causal next token prediction task was investigated. While the derivations should generalize to other domains, both bidirectional and causal, further investigation is necessary to ensure this generalization. Future work can also incorporate changes from these recurrent models to softmax attention to potentially improve efficiency or accuracy of softmax attention implementations.

4 Methods and Results - Reduced Form of Mamba

4.1 Mamba-2

Several approaches have attempted to improve the accuracy of linear attention while retaining linear complexity, such as performer (Choromanski et al., 2021), cosformer (Qin et al., 2022), hedgehog (Zhang et al., 2024), RetNet (Sun et al., 2023), gated linear (Yang et al., 2024a), and DeltaNet (Yang et al., 2024b), among others. One algorithm that is both efficient and improves linear attention significantly is Mamba-2 (Dao and Gu, 2024). In the description below, we examine Mamba-2 purely from an architectural perspective, not from the efficiency gained from the associative scan algorithm.

Algorithm 1 Linear attention

Require: $X \in \mathbb{R}^{N,d}$

Require: $W_{QKV} \in \mathbb{R}^{d,3 \cdot (H \cdot d_h)}$

Require: $W_{out} \in \mathbb{R}^{(H \cdot d_h),d}$

Ensure: $\phi(\cdot) = \text{ReLU}(\cdot) -$
(or any other kernel)

$$\begin{aligned}
 QKV &= XW_{QKV} && \in \mathbb{R}^{N,3 \cdot (H \cdot d_h)} \\
 Q, K, V &= \text{split}(QKV) && \in \mathbb{R}^{H,N,d_h} \\
 y &= ((\phi(Q)\phi(K)^T \odot M)V) && \in \mathbb{R}^{H,N,d_h} \\
 N &= \sum_j (\phi(Q)\phi(K)^T \odot M) && \in \mathbb{R}^{H,d_h} \\
 y_n &= y/N && \in \mathbb{R}^{H,N,d_h} \\
 y'_n &= \text{vec}(y_n)_{(H,N,d_h) \rightarrow (N,H \cdot d_h)} && \in \mathbb{R}^{N,H \cdot d_h} \\
 o &= y'_n W_{out} && \in \mathbb{R}^{N,d}
 \end{aligned}$$

$$\begin{aligned}
 Q &= XW_Q \quad K = XW_K \quad V = XW_V \\
 O &= \frac{[\phi(Q)\phi(K)^T] V}{\sum_j \phi(Q)\phi(K)^T} \quad (24)
 \end{aligned}$$

$$= \frac{\phi(Q) [\phi(K)^T V]}{\sum_j \phi(Q)\phi(K)^T} \quad (25)$$

The original Mamba (Gu and Dao, 2024) architecture was derived from a state space model (SSM) (Kalman, 1960) (Gu et al., 2022), which are not expressive on their own due to having time-independent queries, keys, and values (referred to as C, and B, and x in the SSM literature). Mamba improves SSM expressiveness by making the query, key, and value matrices time-dependent. Mamba-2 further improves efficiency and expressiveness by introducing the associative scan algorithm and decay mask (A mask). The resulting algorithm can be effectively formulated as linear attention with a decay mask (section 4.2 expands this assertion and isolates elements of Mamba-2 that are most expressive). From this perspective, Mamba-2 takes linear attention and makes it more expressive through clever architectural elements and parallelization via associativity.

The complete Mamba-2 algorithm is found in GitHub⁶ and has some additions not described fully in the paper (Dao and Gu, 2024). That is, the Mamba-2 codebase builds in several inductive biases. The full Mamba model with all biases included from this codebase is written out in algorithms 2 and 3. Note that the head and sequence dimensions are arbitrarily swapped to simplify the presented algorithm, though the swaps can be inferred via the explicit shapes. To make the comparison with other algorithms easier, as noted in (Dao and Gu, 2024), we employ the Q , K , and V notation, rather than the equivalent C , B , and x notation (respectively).

Algorithm 2 Mamba-2 Setup

$$\begin{aligned}
 H &= \text{num heads}, \quad d_h = \text{head dim}, \quad N = \text{seq. length}, \quad \mathcal{U}[\cdot] = \text{Uniform Dist.} \\
 d_h &= \text{inner dim}, \quad d_{ssm} = H * d_h \\
 d_{conv} &= 3 * d_{ssm}, \quad dt_{min} = 0.001, \quad dt_{max} = 0.1, \quad l_{min} = \log(dt_{min}), \quad l_{max} = \log(dt_{max}) \\
 A_{log} &= \log(-\mathcal{U}[1, 16]) \in \mathbb{R}^H, \quad D = \text{ones}(H * d_h) \in \mathbb{R}^{H \cdot d_h} \\
 dt_{init} &= \exp(x + l_{min}) \quad \text{where} \quad x \sim \mathcal{U}[-l_{min}, l_{max} - l_{min}] \\
 dt_{bias} &= \text{softplus}^{-1}(dt_{init}) = dt_{init} + \log(-1 + \exp(-dt_{init})) \in \mathbb{R}^H[-4pt]
 \end{aligned}$$

As seen in Algorithm 2 and Algorithm 3, Mamba-2 has numerous elements beyond just a decay mask. Many of the choices seem arbitrary or underexplored and add complexity without much investigation or motivation. We question if the added complexity is necessary for Mamba-2 to be more accurate vanilla linear attention. To analyze the components that make Mamba-2 superior (and propose changes), we break Mamba-2 into its components and build up a simplified algorithm, which we name Mamba-2S, from these singular components.

4.2 Isolating Mamba-2 Accuracy Gains

To investigate the accuracy gains for each element in Mamba-2, we perform several ablations. For all our ablations, we start with the base Llama 2 (Touvron et al., 2023) model and replace the softmax attention block with the Mamba-2

⁶<https://github.com/state-spaces/mamba>

Algorithm 3 Mamba-2 Forward Pass

Require: $h \in \mathbb{R}^{N,d}$, $W_z \in \mathbb{R}^{d,H \cdot d_h}$, $W_{QKV} \in \mathbb{R}^{d,d_{conv}}$ $W_{dt} \in \mathbb{R}^{d,H}$
 $W_{out} \in \mathbb{R}^{H \cdot d_h,d}$, $D \in \mathbb{R}^{H \cdot d_h}$, $A_{log} \in \mathbb{R}^H$, $dt_{bias} \in \mathbb{R}^H$

$Q, K, V = \sigma(\text{conv_1d}(h \cdot W_{QKV}))$	$\in \mathbb{R}^{H,N,d_h}$
$dt_P = h \cdot W_{dt}, \quad Z = h \cdot W_z$	$\in \mathbb{R}^{H,N}$
$A = -\exp(A_{log})$	$\in \mathbb{R}^H$
$dt = \text{softplus}(dt_P + dt_{bias})$	$\in \mathbb{R}^{H,N}$
$D_{res} = V \odot D, \quad V_{dt} = V \odot dt$	$\in \mathbb{R}^{H,N,d_h}$
$A^{CS} = \text{cumsum}(A \odot dt)$	$\in \mathbb{R}^{H,N}$
$A^M = \exp(A^{CS} - (A^{CS})^T) \quad \text{such that } A_{ij}^M = \exp(A_i^{CS} - A_j^{CS})$	$\in \mathbb{R}^{H,N,N}$
$M_{ij} = \begin{cases} 1, & \text{if } i \geq j \\ 0, & \text{if } i < j \end{cases}$	$M \in \mathbb{R}^{H,N,N}$
$y = (QK^T \odot A^M \odot M) \cdot V_{dt}$	$\in \mathbb{R}^{H,N,d_h}$
$y_D = y + D_{res}$	$\in \mathbb{R}^{H,N,d_h}$
$y'_D = \text{vec}(y_D)_{(H,N,d_h) \rightarrow (N,H \cdot d_h)}$	$\in \mathbb{R}^{N,H \cdot d_h}$
$y_N = \text{RMSNorm}(y'_D \odot \sigma(Z))$	$\in \mathbb{R}^{N,H \cdot d_h}$
$out = y_N \cdot W_{out}$	$\in \mathbb{R}^{N,d}$

block. Our small model is about 300 million parameters. We use this model for all ablations, which helps to facilitate rapid experimentation. The medium model is about 700 million parameters and is used after ablation experiments to examine scalability of the algorithm. The maximum sequence length of all ablations is 2048 unless specified. For comparisons and reporting, we plot the test loss over 90K steps⁷ and the final test loss at the end of training. Each model is trained to perform next token language modeling using the HuggingFace FineWeb dataset (CC-MAIN-2024-51 version) (Penedo et al., 2024). This dataset is composed of over 15 Trillion clean and deduplicated tokens from CommonCrawl, a dataset of a crawl of webpages on the internet. As such, it is composed of various different types of data, from code, to random documents. Mamba-2 (Dao and Gu, 2024) was shown to be an expressive, powerful

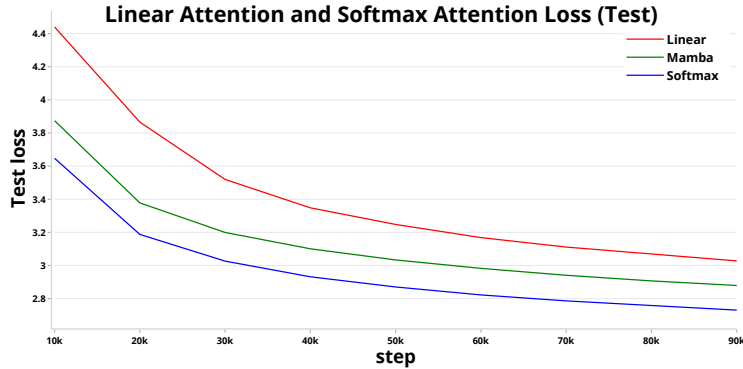


Figure 12: Accuracy of linear attention, Mamba, and softmax attention, keeping everything but the attention mechanism constant across experiments.

architecture. One that is both fast and accurate. Figure 12 shows the test loss for Mamba-2, normal linear attention, and softmax attention. As seen in Figure 12, Mamba-2 is significantly better than normal linear attention and is much closer in accuracy to softmax attention. However, most elements in the codebase were not ablated, leaving to question which

⁷Our code trains for 100K steps and evaluates on the test loss every 10K steps. However, a bug exists which does the evaluation on 1 step after each 10K steps, not evaluating the 100K'th step. We do not re-evaluate all experiments as it would take months of retraining and we believe 90K $\rightarrow$ 100K would not lead to any insights.

components are important to the expressive power of Mamba-2 and which are unnecessary. We examined the official Mamba-2 repository ⁸ and isolate the components of the Mamba-2 block. Specifically, we ablate the following ⁹:

1. QK activation type (SiLU, ReLU or None)
2. A -mask type (Original or Softplus)
3. Input convolution window size (window size one (no convolution), two, three, or four)
4. Additive D residual (binary, present or not)
5. Multiplicative Z gate (binary, present or not)
6. Normalization type (Output or Softmax)
7. Value discretization, dt (binary, present or not)

While most of the ablations performed are straightforward, however the normalization type and A -mask type should be highlighted. This work calls normalization on the QK inner product *softmax normalization*. Linear attention with a strictly positive inner product space can use softmax normalization, but Mamba-2 cannot as the queries and keys are not necessarily positive. Instead, Mamba-2 uses an RMS norm layer before the output projection. We call this form of normalization *output normalization*. The A -mask in the original Mamba-2 paper is tied to the values by the discretization parameter, which we call *original A -masking*. We ablate if associating the A -mask with the discretization parameter is necessary. Because the A -mask must be strictly negative, as shown in equation (26), we test the negative softplus function on the A -mask, remove the discretization parameter and name this *softplus A -masking*.

$$\begin{aligned} \text{"original" } A\text{-masking} &\rightarrow A = -\exp(A_{log}) \odot dt \\ \text{"softplus" } A\text{-masking} &\rightarrow A = -\text{softplus}(A) \end{aligned} \quad (26)$$

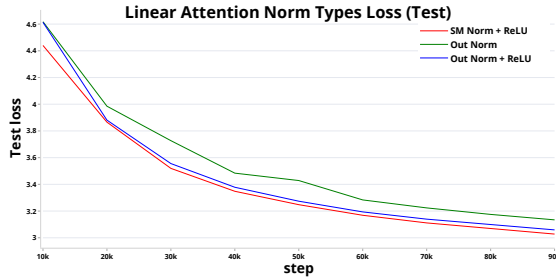


Figure 13: Accuracy of various norm types. Softmax normalization requires a positive inner-product space image, as such we use ReLU.

Model	Train loss	Test loss
ReLU	3.12	3.06
Conv (w=2)	2.87	2.93
Conv (w=3)	2.94	2.91
Conv (w=4)	2.84	2.9
Conv (w=2) + SiLU	2.95	2.9
Conv (w=3) + SiLU	2.95	2.89
Conv (w=4) + SiLU	2.9	2.89

Table 2: Adding a convolution to normal linear attention increases accuracy.

To begin our ablation study, we need a form of linear attention that will allow for more diverse experimentation. The original linear attention formulation as proposed by (Katharopoulos et al., 2020) has softmax-style normalization on the QK inner product. This normalization restricts experimentation to a strictly non-negative inner product function range. Stability aside, as seen in Figure 13, adopting an output norm is nearly as accurate as softmax norm for basic linear attention. When ablating components of Mamba-2, we therefore use an output norm as it does not necessitate a strictly positive inner product space pre-image, giving more freedom to perform similar ablations. ¹⁰

The first element of Mamba-2 we examine is the input convolution. As seen in Table 2, adding a convolution with kernel size 2 is significantly better than normal linear attention. Increasing the convolution size to 3 or adding an activation function (SiLU used in Mamba) gives a slight accuracy gain, but is not as impactful as adding the convolution itself. We only test up to a convolution size of 4 as the implementation of `causal_conv_1d`¹¹ only allows for a max window size of 4. As the window sizes increases beyond a window size of 2, the accuracy gains are small. To keep the implementation minimal, we adopt a convolution of window size 2 without an activation function. A window size of 2 minimizes the memory necessary in the complete attention operation. Increasing the convolution window requires storing $3d$ more memory per window size increase. That is, storing an additional past token for each of the queries, keys, and values. As

⁸<https://github.com/state-spaces/mamba>

⁹Additional ablations found in our codebase. These are the most notable and interesting.

¹⁰Since linear attention with QK normalization requires a strictly positive image of the QK inner product, some sort of activation function must be used. As such, we cannot test *SM Norm* by itself.

¹¹<https://github.com/Dao-AI-Lab/causal-conv1d>

for the activation function, the training loss appears to increase while the test loss decreases when adding the activation function. We opt to remove the activation function for simplicity, however adding it appears to slightly improve model test loss while increasing train loss.

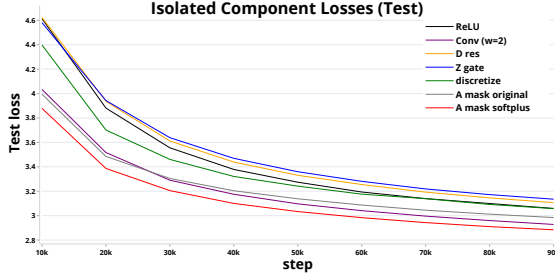
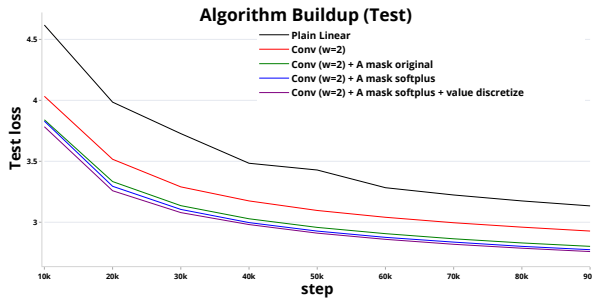


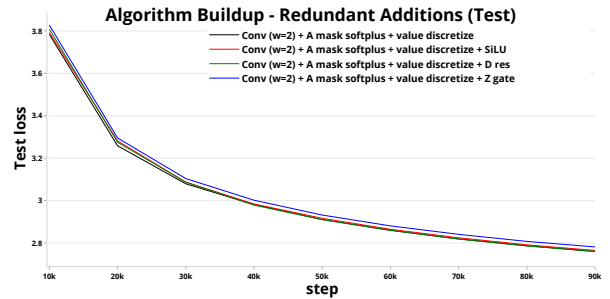
Figure 14: Isolated Mamba ablation

Model	Train loss	Test loss
SM Norm + ReLU	3.0	3.03
Out Norm	3.19	3.13
Out Norm + ReLU	3.12	3.06
Out Norm + Conv (w=2)	2.87	2.93
Out Norm + Conv (w=3)	2.94	2.91
Out Norm + Conv (w=2) + SiLU	2.95	2.9
Out Norm + D res	3.18	3.11
Out Norm + Z gate	3.2	3.13
Out Norm + value discretize	3.06	3.06
Out Norm + A-mask original	2.94	2.98
Out Norm + A-mask softplus	2.85	2.88

Table 3: Table of isolated component additions added to linear attention.



(a) Main buildup



(b) Redundant components after buildup

Figure 15: Investigations of major and minor build ups in constructing the simplified Mamba-2 architecture.

We ablate the rest of the components of Mamba mentioned above in isolation. All isolated ablation test values can be found in Table 3. Figure 14 also shows the results for ablating each component in isolation, graphically. The most impactful component was the *A*-mask, specifically the softplus variant. The original *A*-mask variant used in Mamba is still quite impactful on model accuracy, but not as much as the softplus variant. The second most impactful component is the convolution. The rest of the isolated components result in minor accuracy gains compared to vanilla linear attention. We note that, of the collection of minor improvements, the value discretization method is most impactful. Even so, its impact is eclipsed by the *A*-mask and convolution components.

4.3 Building Up to the Mamba-2S Base Model

With each isolated component ablated, we want to buildup a minimal version of Mamba-2 that is as accurate as the original Mamba-2 algorithm (see Table 4). We start with the *output norm* variant of linear attention and build up the model on top of this base. While investigating all component interactions would be desirable, the number of combinations is too numerous. Instead, we build up the model using insights from the isolated component ablations. Figure 15a shows the main model buildup. To build up the new model, we start by combining the convolution and softplus *A*-mask as these two isolated components had the largest impact on accuracy, as shown in Figure 14. We find the combination of these two components results in higher accuracy than the isolated components themselves and use this as the new base model. We then investigate adding a third component, finding that adding the time discretization parameter gives a small boost in accuracy. In Figure 15b, we show that adding other components do not help increase accuracy of the model. In fact, adding a *Z* gate slightly reduces accuracy of the model. All buildup investigations can be found in Table 4.

To keep the model both simple and accurate, our resulting method uses the softplus A -mask, an input convolution of size 2, and time discretization. This simplified Mamba-2 algorithm, which we call Mamba-2S, only retains necessary components as shown in Algorithm 4.

Model	Train loss	Test loss
Plain Linear	3.19	3.13
Conv (w=2)	2.87	2.93
Conv (w=2) + A -mask original	2.81	2.8
Conv (w=2) + A -mask softplus	2.67	2.77
<i>Conv (w=2) + A-mask softplus + value discretize</i>	2.7	2.76
Conv (w=2) + A -mask softplus + value discretize + SiLU	2.73	2.76
Conv (w=2) + A -mask softplus + value discretize + D res	2.79	2.76
Conv (w=2) + A -mask softplus + value discretize + Z gate	2.72	2.78

Table 4: Table of all additions (buildups) leading up to our final algorithm. Italics convey the components in 2Mamba.

Algorithm 4 Proposed Mamba-2 Simplified Forward (Mamba-2S): no setup besides weights

Require: $h \in \mathbb{R}^{N,d}$, $W_{QKV} \in \mathbb{R}^{d,3 \cdot H \cdot d_h}$, $W_{dt} \in \mathbb{R}^{d,H}$, $W_A \in \mathbb{R}^{d,H}$, $W_{out} \in \mathbb{R}^{H \cdot d_h,d}$

$$\begin{aligned}
 Q, K, V &= \text{conv_1d}(h \cdot W_{QKV}) && \in \mathbb{R}^{H,N,d_h} \\
 dt &= \text{softplus}(h \cdot W_{dt}) && \in \mathbb{R}^{H,N} \\
 A &= -\text{softplus}(h \cdot W_A) && \in \mathbb{R}^{H,N} \\
 V_{dt} &= V \odot dt && \in \mathbb{R}^{H,N,d_h} \\
 A^{CS} &= \text{cumsum}(A) && \in \mathbb{R}^{H,N} \\
 A^M &= \exp(A^{CS} - (A^{CS})^T) \quad \text{where } A_{ij}^M = \exp(A_i^{CS} - A_j^{CS}) && \in \mathbb{R}^{H,N,N} \\
 M_{ij} &= \begin{cases} 1, & \text{if } i \geq j \\ 0, & \text{if } i < j \end{cases} && M \in \mathbb{R}^{H,N,N} \\
 y &= (QK^T \odot A^M \odot M) \cdot V_{dt} && \in \mathbb{R}^{H,N,d_h} \\
 y' &= \text{vec}(y)_{(H,N,d_h) \rightarrow (N,H \cdot d_h)} && \in \mathbb{R}^{N,H \cdot d_h} \\
 y_N &= \text{RMSNorm}(y') && \in \mathbb{R}^{N,H \cdot d_h} \\
 out &= y_N \cdot W_{out} && \in \mathbb{R}^{N,d}
 \end{aligned}$$

4.4 Second Order

While Mamba-2S is quite strong in accuracy as seen in Figure 16a, it still falls short of softmax attention. (Mongaras and Larson, 2025) showed that squaring the QK inner product results in a significant accuracy boost. Additionally, squaring the QK inner product results in a positive inner product space image. This space can adopt an online normalization algorithm like *softmax normalization*, which is more stable than *output normalization*. Using the simplified Algorithm 4, we find that squaring the QK inner product does result in a significant accuracy gain. As seen in Figure 16a, squaring the inner product results in softmax level accuracy while being more memory efficient than softmax attention for sufficient sequence length. Specifically, from Mamba-2S 4, we remove the RMSNorm, add softmax normalization, and replace the QK inner product with a squared inner product. We opt to use softmax normalization over output normalization as softmax normalization is more stable when using the online softmax algorithm (Milakov and Gimelshein, 2018). Online softmax normalization, like Flash Attention (Dao et al., 2022), requires an exponential to update the max statistic over blocks. As the A -mask has an exponential, online softmax-like normalization can still be effective when the max statistic is stored in the A -mask as opposed to the e^{QK} inner product, as used in online softmax. The gradients for the forward and backward pass are supplied in Appendix D. Full Triton kernels can be found in our codebase, which are necessary for training.

The time discretization operation does improve the model, however the accuracy gains are minimal compared to adding the convolution and A -mask. For smaller models, training is stable. For the medium model, we found that the magnitude of the values significantly increased across update steps, leading to numerical instability and divergence in training as seen in Figure 16b. Training a larger model with discretization can be somewhat stabilized—however this requires

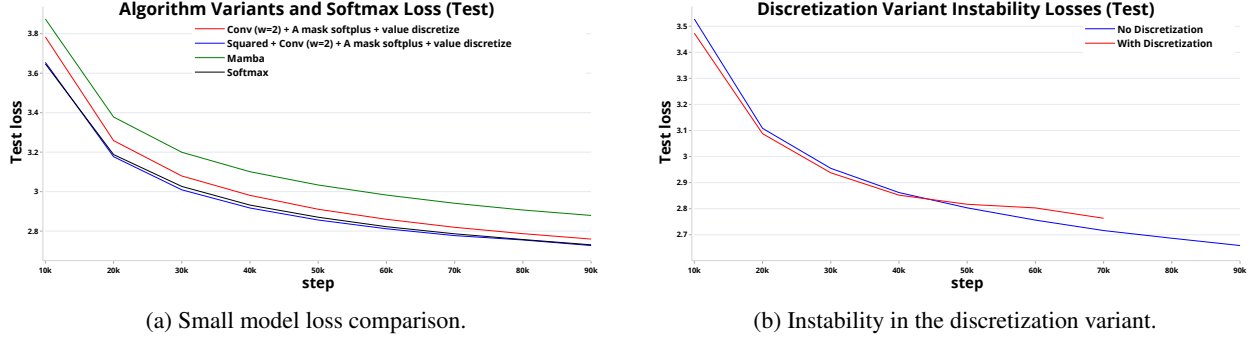


Figure 16: Experimental results comparing accuracy and training stability.

forcing triton to use IEEE FP32 input precision which is about 8x slower (NVIDIA Corporation, 2020) than the TF32 precision (a reduced precision mantissa compared to the standard IEEE FP32). A reasonable middle ground is a mix of input precision of FP32x3 for dot operations that need more precision and TF32 for others, which is still slower than full TF32 precision but is slightly more stable. Training in pure FP32 PyTorch also results in stable training, but is considerably slower. While manipulating the kernel can result in more stable training, to keep the squared variant numerically stable for low precision kernels, we remove the time discretization operation for the medium model.

Our proposed squared algorithm, which we name 2Mamba, can be found in Algorithm 5. As seen in Figure 17, across various model sizes and sequence lengths, 2Mamba is nearly as accurate as softmax attention. One aspect worth further discussion is the placement of the square operation. We specifically place it on the QK inner product to increase the inner product dimension. An alternative placement could also include the A -mask. However, we find that including the A -mask does not result in any accuracy gains. While squaring after the A -mask application does result in a squared inner product on Q and K , it also results in squared terms in the A -mask, which is unnecessary. Intuitively, the A -mask should be applied directly to attention scores, which come from the squared inner product of Q and K .

Algorithm 5 2Mamba Algorithm

Require: $h \in \mathbb{R}^{N,d}$, $W_{QKV} \in \mathbb{R}^{d,3 \cdot H \cdot d_h}$, $W_A \in \mathbb{R}^{d,H}$, $W_{out} \in \mathbb{R}^{H \cdot d_h,d}$

$$\begin{aligned}
 Q, K, V &= \text{conv_1d}(h \cdot W_{QKV}) && \in \mathbb{R}^{H,N,d_h} \\
 A &= -\text{softplus}(h \cdot W_A) && \in \mathbb{R}^{H,N} \\
 A^{CS} &= \text{cumsum}(A) && \in \mathbb{R}^{H,N} \\
 A^M &= \exp(A^{CS} - (A^{CS})^T) \quad \text{where} \quad A_{ij}^M = \exp(A_i^{CS} - A_j^{CS}) && \in \mathbb{R}^{H,N,N}, \\
 M_{ij} &= \begin{cases} 1, & \text{if } i \geq j \\ 0, & \text{if } i < j \end{cases} && M \in \mathbb{R}^{H,N,N} \\
 y &= ((QK^T)^2 \odot A^M \odot M) \cdot V && \in \mathbb{R}^{H,N,d_h} \\
 N &= \sum_j ((QK^T)^2 \odot A^M \odot M) && \in \mathbb{R}^{H,d_h} \\
 y_N &= y/N && \in \mathbb{R}^{H,N,d_h} \\
 y'_N &= \text{vec}(y_N)_{(H,N,d_h) \rightarrow (N,H \cdot d_h)} && \in \mathbb{R}^{N,H \cdot d_h} \\
 out &= y'_N \cdot W_{out} && \in \mathbb{R}^{N,d}
 \end{aligned}$$

4.5 2Mamba Algorithm Efficiency

While squaring the inner product does result in a large accuracy gain, we note that the recurrent state requires approximately the square of the space as the original linear variant. When squaring the inner product or taking the Kronecker product of Q and K , and then performing the inner product, some terms are repeated due to the commutativity of the multiplication operation in the inner product. This means that as opposed to doing a Kronecker product on the queries and keys, one can do an operation that calculates all unique second-order product terms and calculate their

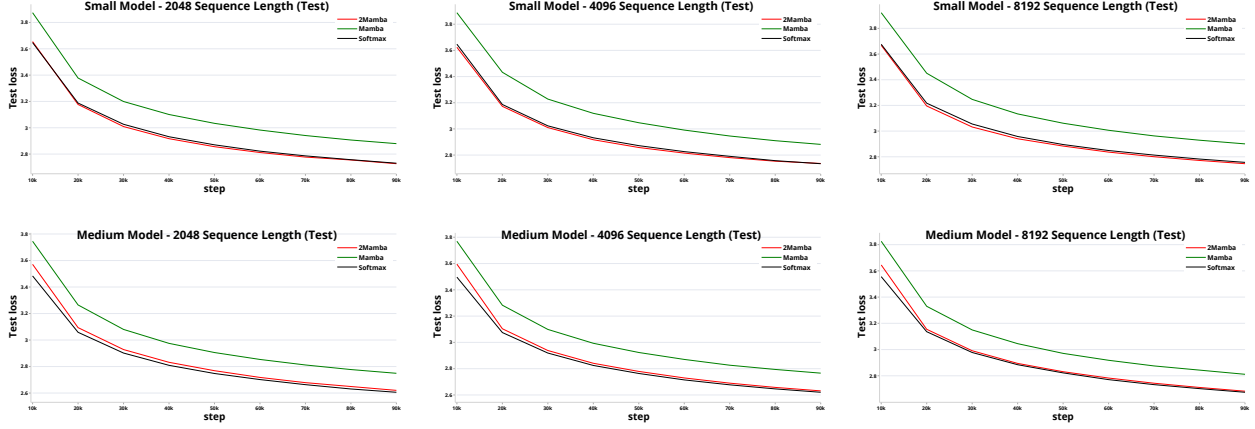


Figure 17: **First row:** Small model ($\sim 300\text{M}$ params) test loss for Mamba, 2Mamba, and softmax attention trained on 2048, 4096, and 8192 sequence lengths. **Second row:** Medium model ($\sim 700\text{M}$ params) test loss for Mamba, our proposed algorithm, and softmax attention trained on 2048, 4096, and 8192 sequence lengths

multiplicity via the multinomial theorem. We provide a Triton kernel for creating a vector of second order products of a given vector.¹² The number of terms, as seen in equation (27) is equal to $\frac{d(d+1)}{2}$, which is less than half the elements within the vector obtained from naively calculating all d^2 elements.¹³

$$num_terms = \binom{2+d-1}{d-1} = \binom{d+1}{d-1} = \frac{(d+1)!}{2(d-1)!} = \frac{(d+1) \cdot d \cdot (d-1)!}{2(d-1)!} = \frac{d \cdot (d+1)}{2} \quad (27)$$

As the complexity of softmax attention is linear in memory due to the KV cache, there exists a sequence length such that squaring the inner product requires less memory than softmax attention. A KV cache requires memory on the order of $2Ld$ for a single attention head. By using equation (27) for the key dimension and multiplying by d for the value dimension, a hidden state for squared linear attention requires memory on the order of $d^2(d+1)/2$ per attention head. Adding a convolution adds an extra $3d$ memory per attention head and using softmax normalization adds an extra $d(d+1)/2$ memory per attention head. In total, this results in a hidden state size of $\frac{d(d+1)^2}{2} + 3d$ per attention head. The inequality in equation (28) provides the necessary sequence length, N , such that any sequence length longer than N is more memory efficient using a second-order hidden state than using softmax attention.

$$2Nd > \frac{d(d+1)^2}{2} + 3d \quad \rightarrow \quad N > \frac{(d+1)^2}{4} + \frac{3}{2} \quad (28)$$

In practice, we used a head dimension of 64. According to inequality in equation (28), a sequence length greater than $\frac{(64+1)^2}{4} + \frac{3}{2} \approx 1058$ would result in a lower memory usage than softmax attention. To ensure the squared model retains good accuracy beyond this sequence length, we examine models with a context size up to 8192 tokens. As such, squaring the QK inner product is less memory constrained than softmax attention past the sequence length of 1058 while matching softmax level accuracy. This memory result is verified by running both softmax attention and 2Mamba while accumulating the raw hidden state memory usage from each model, as seen in figure 18. The codebase has a state caching inference implementation for both algorithms and supply pseudocode in 6. Inference requires keeping around a few hidden states. $H_{t-1}^\uparrow$ is the numerator, keeping all key-value products. $H_{t-1}^\downarrow$ is the denominator, only keeping the keys and is only necessary for softmax-like normalization. With an input convolution, one must keep around $window_size - 1$ number of queries, keys, and values. The code below shows a window size of 2, so only one query, key, and value must be cached.

¹²<https://github.com/gmongaras/Triton-Efficient-Kronecker-Product>

¹³For notational brevity, we denote the head dimension as d in this section.

Algorithm 6 2Mamba Inference

Require: $h \in \mathbb{R}^d$, $W_{QKV} \in \mathbb{R}^{d, 3 \cdot (H \cdot d_h)}$, $W_A \in \mathbb{R}^{d, H}$, $W_{out} \in \mathbb{R}^{H \cdot d_h, d}$

Require: $H_{t-1}^\uparrow \in \mathbb{R}^{H, D, d_h}$, $H_{t-1}^\downarrow \in \mathbb{R}^{H, D}$, $q_{t-1} \in \mathbb{R}^{H, d_h}$, $k_{t-1} \in \mathbb{R}^{H, d_h}$, $v_{t-1} \in \mathbb{R}^{H, d_h}$

Require: $1 \leq D = \frac{d_h(d_h+1)}{2} \leq d_h^2$

$$\begin{aligned}
 q_t, k_t, v_t &= h_t W_{QKV} && \in \mathbb{R}^{H, d_h} \\
 q_t^c, k_t^c, v_t^c &= \text{conv_1d}(q_t, k_t, v_t, q_{t-1}, k_{t-1}, v_{t-1}, \text{window_size} = 2) && \in \mathbb{R}^{H, d_h} \\
 a_t &= \exp(-\text{softplus}(h \cdot W_A)) && \in \mathbb{R}^H \\
 (q_t^c)^{\otimes 2} &= q_t^c \otimes q_t^c && \in \mathbb{R}^{H, D} \\
 (k_t^c)^{\otimes 2} &= k_t^c \otimes k_t^c && \in \mathbb{R}^{H, D} \\
 H_t^\uparrow &= H_{t-1}^\uparrow \odot a_t + [(k_t^c)^{\otimes 2}]^T \cdot (v_t^c) && \in \mathbb{R}^{H, D, d_h} \\
 H_t^\downarrow &= H_{t-1}^\downarrow \odot a_t + [(k_t^c)^{\otimes 2}]^T && \in \mathbb{R}^{H, D} \\
 y_N &= \frac{q_t \cdot H_t^\uparrow}{q_t \cdot H_t^\downarrow} && \in \mathbb{R}^{H, d_h} \\
 o_t &= y_t \cdot W_{out} && \in \mathbb{R}^d
 \end{aligned}$$

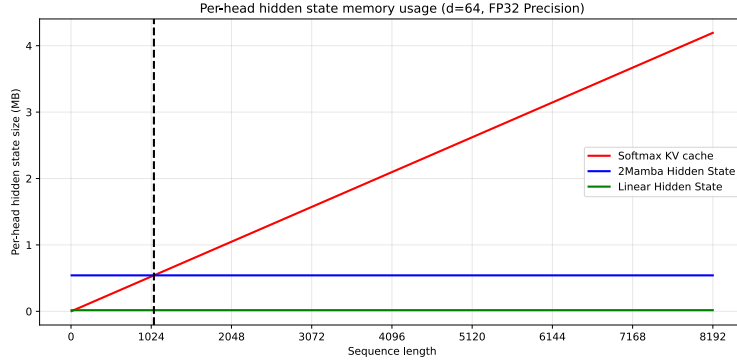


Figure 18: Memory usage of a single head of softmax attention, 2Mamba, and linear attention, utilizing softmax-like normalization and a convolution with kernel size 2.

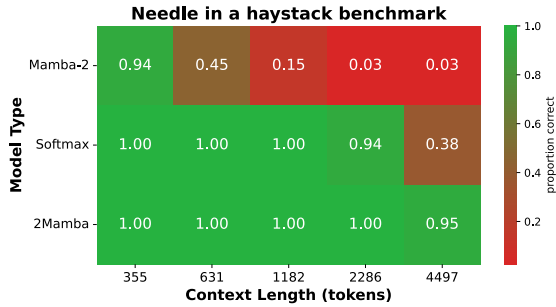


Figure 19: One-shot NIAH benchmark results using maximum likelihood sampling over 1000 sequences. Green indicates the predicted needle was correct more often while red indicates the predicted needle was predicted less often. Scores are indicative of the proportion of sequences in which the model predicted the needle correctly.

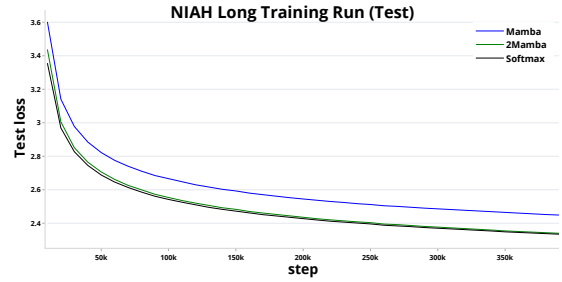


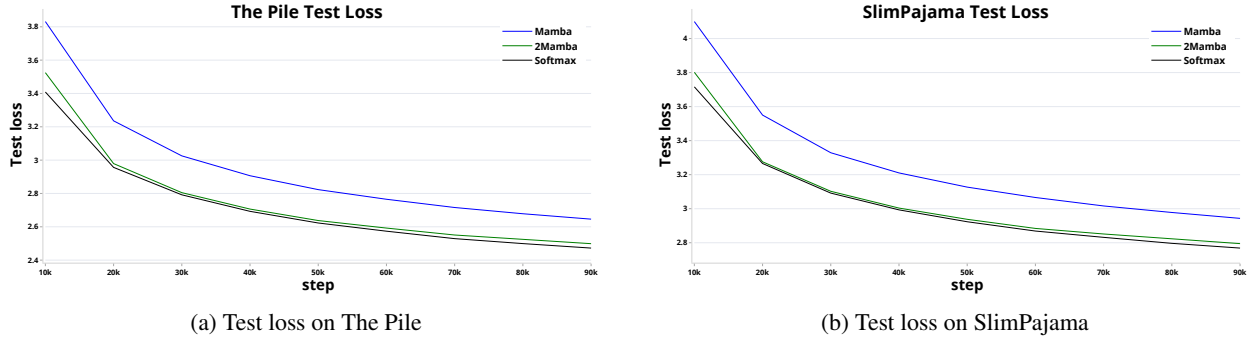
Figure 20: Long training run of various models with a batch size of 64 on 8,192 max sequence length.

4.6 Effective Context Usage

To confirm 2Mamba uses its context effectively, we employ the The Needle in a Haystack (NIAH) test, which measures ability of the model to retrieve a specific, relevant fact (the “needle”) inserted into a long prompt (the “haystack”). That is, it assesses how well a model recalls information placed at different locations within large prompts. In our testing, we train a model for 400,000 steps on a batch size of 64 with a maximum context size of 8,192 tokens. During this long training run, we plot the test loss, as seen in Figure 20, showing that 2Mamba is still competitive with softmax attention for long training runs. After training for 400,000 steps, we evaluate our model on Nanotron’s needle in a haystack test¹⁴ on context lengths from about 1,024 characters to about 16,384 characters. This test hides a password inside a large corpus of irrelevant text. The model must memorize the password, and repeat it after parsing the large corpus of text. This benchmark tests how well a model uses its context window and for how long it can store important information. As seen in Figure 19, 2Mamba method is slightly better than softmax in context retrieval, and much better than Mamba-2. This result provides evidence that 2Mamba extends beyond NTP tasks and can utilize its context effectively.

4.7 Additional Results

To verify 2Mamba generalizes to other datasets, we perform a training run with medium sized models on The Pile (Gao et al., 2021) and SlimPajama (Shen et al., 2023).



4.8 2Mamba With an Exponentiated Hidden State

The property of squaring the inner product was used to make Mamba-2 more accurate while remaining efficient. As shown in (Mongaras and Larson, 2025), adding higher orders typically results in additional accuracy gains (though diminishing with each new term). In the limiting case, one can use the exponential function which results in softmax attention. Therefore, we ask: If the exponential function on normal linear attention gives softmax attention, then what would happen if the QK inner product of 2Mamba 4 was exponentiated? As seen in Figure 22, exponentiating

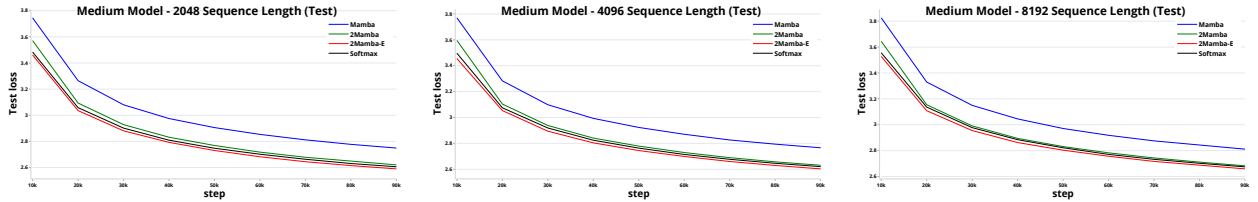


Figure 22: Medium model (~700M params) test loss for Mamba, 2Mamba-E, and softmax attention trained on 2048, 4096, and 8192 sequence lengths.

the inner product of Q and K results in slightly better accuracy than softmax attention. We name the exponentiated form 2Mamba-E. One can interpret this algorithm as either exponentiating 2Mamba or as softmax attention with an A -mask and input convolution. The latter explanation seems more intuitive and satisfying when considering traditional transformer literature. Adding an A -mask, or decay mask, to softmax attention has also been proposed in the Forgetting Transformer (Lin et al., 2025). The main difference between the forgetting transformer and the algorithm we use is the

¹⁴https://huggingface.co/datasets/nanotron/simple_needle_in_a_hay_stack

A-mask construction. The forgetting transformer uses $\log(\text{sigmoid}(x))$ as the function for the *A*-mask values while we use $-\text{softplus}(-x)$. It turns out these are nearly equivalent (only differing by sign) as seen in equation 29.

$$\log(\text{sigmoid}(x)) = \log\left(\frac{1}{1 + e^{-x}}\right) = -\log(1 + e^{-x}) = -\text{softplus}(-x) \quad (29)$$

From this perspective, our work on exponentiated hidden states also provides an intuitive connection between SSM models, like Mamba-2, and transformer variants like the forgetting transformer.

4.9 2Mamba Conclusions

This work examines the importance of each individual component of Mamba-2 to build up a minimal implementation, Mamba-2S. From this base model, we take the second order hidden state to achieve a model, 2Mamba, that is as accurate as softmax attention, yet constant in memory. Additionally, we show that our implementation, when exponentiated, results in an architecture, 2Mamba-E, that is better than softmax attention and show that this implementation is similar to the forgetting transformer (Lin et al., 2025). 2Mamba is a step forward in bridging the gap between linear attention architectures and softmax attention, while being more efficient in practice.

The exploration of this work closely examined the architecture choices made in Mamba-2 (Dao and Gu, 2024). Another parallel work that improves linear attention is DeltaNet (Yang et al., 2024b). Mamba-2 and Deltanet have been combined in Gated Deltanet (Yang et al., 2025). Similarly, future work can look into adding DeltaNet to 2Mamba to further improve the algorithm.

Future work can also look into the hidden state created by squaring the inner product of the queries and keys. While we stick with a static hidden state size, future work can examine how varying the hidden state size affects downstream accuracy and if squaring the inner product space results in better or similar accuracy than using a query and key projection to the same size. This future work can make 2Mamba more efficient in terms of memory by optimizing the hidden state size.

5 Conclusion and Discussion

Attention may be all you need, but not necessarily softmax attention. This thesis confirms that linear attention and softmax attention do indeed have a large accuracy gap. Understanding this gap is necessary to be able to potentially improve linear attention to softmax attention accuracy in an effective manner. Work done in this thesis shows that softmax attention and linear attention can be related via the way linear attention was first derived, by a kernel function decomposition. The main part of softmax attention is the exponential and row-sum denominator. The exponential does have a kernel function decomposition. By the Taylor expansion of the exponential, it can be shown that the kernel function on Q and K can be viewed as either projecting Q and K into an infinite dimensional space or as a sum of infinite RNNs with higher orders of Q and K by the kronecker or tensor product. Taking the first order term in the expansion yields linear attention. From this perspective on softmax attention, it is obvious why naive linear attention appears bounded by softmax accuracy, as linear attention is a linear approximation of softmax attention. It is also shown that the denominator is not special to softmax, rather it is just a choice of normalization used to the recurrent sum. This work was published in TMLR.¹⁵

Despite the efficiency gain from linear attention, the accuracy discrepancy is too large to practically use naive linear attention for most applications. With an understanding of the linear-softmax accuracy gap, this work attempts to make linear attention more accurate, closer to softmax level accuracy. Instead of starting completely from scratch, this work starts from an accurate baseline linear attention algorithm. Two such algorithms are Mamba2 (Dao and Gu, 2024) and DeltaNet (Yang et al., 2024b). Mamba2 is widely used, a lot more simple than deltanet, and is very strong in accuracy. Additionally, the codebase has several changes that may or may not improve the linear attention algorithm, which is beneficial to explore and ablate. Starting from Mamba2, this work first significantly simplifies the architecture to the most fundamental pieces that make it more accurate than naive linear attention. The *A*-mask function is then improved by changing the activation function. Using insights from the previous section, the hidden state of Mamba2 is squared to obtain a non-negative hidden state that is not excessively large and is more accurate than a linear hidden state. It was also shown that the value gate makes 2Mamba unstable at scale, and as such this activation was removed. This new model is close to being as accurate as softmax attention, while being constant in memory. Additionally, using insights from section 3, the hidden state can be exponentiated to obtain a quadratic model that is more accurate than softmax attention and very similar to the Forgetting Transformer (Lin et al., 2025). A paper based on this section is planned to be submitted to NeurIPS.

¹⁵https://tmlr.infinite-conf.org/paper_pages/PHcITOI3vV.html

There are many directions future work can go. The first direction explores the mathematical side, stemming from section 3. The relationship between linear attention and softmax attention is clear via the Taylor Expansion of the exponential. However, direct mathematical relationships to other models like Mamba2 do not exist, making it difficult to understand why adding specific components of Mamba2 make it better than naive linear attention. Future work can also examine section 4, looking into making a more accurate model. The methods for improving linear attention were performed on the Mamba2 model. However other accurate models such as DeltaNet could also be analyzed to potentially create a linear model that is more accurate than softmax attention. Another interesting direction is applying the methods for making 2Mamba on other modalities such as timeseries and images/video.

References

- Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints, 2023. URL <https://arxiv.org/abs/2305.13245>.
- Yaroslav Aksenov, Nikita Balagansky, Sofia Maria Lo Cicero Vaina, Boris Shaposhnikov, Alexey Gorbatovski, and Daniil Gavrilov. Linear transformers with learnable kernel functions are better in-context models, 2024. URL <https://arxiv.org/abs/2402.10644>.
- Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In Yoshua Bengio and Yann LeCun, editors, *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015. URL <http://arxiv.org/abs/1409.0473>.
- Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. *arXiv preprint arXiv:2501.00663*, 2024.
- Ali Behrouz, Zeman Li, Praneeth Kacham, Majid Daliri, Yuan Deng, Peilin Zhong, Meisam Razaviyayn, and Vahab Mirrokni. Atlas: Learning to optimally memorize the context at test time, 2025. URL <https://arxiv.org/abs/2505.23735>.
- Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv:2004.05150*, 2020.
- Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Tomas Jackson, Sally Jesmonth, Nikhil J Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Isabel Leal, Kuang-Huei Lee, Sergey Levine, Yao Lu, Utsav Malla, Deeksha Manjunath, Igor Mordatch, Ofir Nachum, Carolina Parada, Jodilyn Peralta, Emily Perez, Karl Pertsch, Jornell Quiambao, Kanishka Rao, Michael Ryoo, Grecia Salazar, Pannag Sanketi, Kevin Sayed, Jaspiar Singh, Sumedh Sontakke, Austin Stone, Clayton Tan, Huong Tran, Vincent Vanhoucke, Steve Vega, Quan Vuong, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Tianhe Yu, and Brianna Zitkovich. Rt-1: Robotics transformer for real-world control at scale, 2023. URL <https://arxiv.org/abs/2212.06817>.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc., 2020a. URL https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfc4967418bfb8ac142f64a-Paper.pdf.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020b.
- Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. End-to-end object detection with transformers. In *European conference on computer vision*, pages 213–229. Springer, 2020.
- Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Michael Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. Decision transformer: Reinforcement learning via sequence modeling. In *ICML 2021 Workshop on Unsupervised Reinforcement Learning*, 2021. URL <https://openreview.net/forum?id=gaCGNwsWITG>.
- Yingfa Chen, Zhen Leng Thai, Zihan Zhou, Zhu Zhang, Xingyu Shen, Shuo Wang, Chaojun Xiao, Xu Han, and Zhiyuan Liu. Hybrid linear attention done right: Efficient distillation and effective architectures for extremely long contexts, 2026. URL <https://arxiv.org/abs/2601.22156>.

- Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.
- Kyunghyun Cho, B van Merriënboer, Caglar Gulcehre, F Bougares, H Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. In *Conference on Empirical Methods in Natural Language Processing (EMNLP 2014)*, 2014.
- Krzysztof Marcin Choromanski, Valerii Likhoshesterov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Quincy Davis, Afroz Mohiuddin, Lukasz Kaiser, David Benjamin Belanger, Lucy J Colwell, and Adrian Weller. Rethinking attention with performers. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=Ua6zuk0WRH>.
- Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24(240):1–113, 2023.
- Liam Collins, Advait Parulekar, Aryan Mokhtari, Sujay Sanghavi, and Sanjay Shakkottai. In-context learning with transformers: Softmax attention adapts to function lipschitzness. *Advances in Neural Information Processing Systems*, 37:92638–92696, 2024.
- Katherine Crowson, Stefan Andreas Baumann, Alex Birch, Tanishq Mathew Abraham, Daniel Z Kaplan, and Enrico Shippole. Scalable high-resolution pixel-space image synthesis with hourglass diffusion transformers. *arXiv preprint arXiv:2401.11605*, 2024.
- Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*, 2023.
- Tri Dao and Albert Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. In Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp, editors, *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 10041–10071. PMLR, 21–27 Jul 2024. URL <https://proceedings.mlr.press/v235/dao24a.html>.
- Tri Dao, Daniel Y Fu, Stefano Ermon, Atri Rudra, and Christopher Re. Flashattention: Fast and memory-efficient exact attention with IO-awareness. In Alice H. Oh, Alekh Agarwal, Danielle Belgrave, and Kyunghyun Cho, editors, *Advances in Neural Information Processing Systems*, 2022. URL <https://openreview.net/forum?id=H4DqfPSibmx>.
- DeepSeek-AI, Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenhao Xu, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Erhang Li, Fangqi Zhou, Fangyun Lin, Fucong Dai, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Hao Li, Haofen Liang, Haoran Wei, Haowei Zhang, Haowen Luo, Haozhe Ji, Honghui Ding, Hongxuan Tang, Huanqi Cao, Huazuo Gao, Hui Qu, Hui Zeng, Jialiang Huang, Jiashi Li, Jiaxin Xu, Jiewen Hu, Jingchang Chen, Jingting Xiang, Jingyang Yuan, Jingyuan Cheng, Jinhua Zhu, Jun Ran, Junguang Jiang, Junjie Qiu, Junlong Li, Junxiao Song, Kai Dong, Kaige Gao, Kang Guan, Kexin Huang, Kexing Zhou, Kezhao Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Wang, Liang Zhao, Liangsheng Yin, Lihua Guo, Lingxiao Luo, Linwang Ma, Litong Wang, Liyue Zhang, M. S. Di, M. Y Xu, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingxu Zhou, Panpan Huang, Peixin Cong, Peiyi Wang, Qiancheng Wang, Qihao Zhu, Qingyang Li, Qinyu Chen, Qiushi Du, Ruiling Xu, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runqiu Yin, Runxin Xu, Ruomeng Shen, Ruoyu Zhang, S. H. Liu, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaofei Cai, Shaoyuan Chen, Shengding Hu, Shengyu Liu, Shiqiang Hu, Shirong Ma, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, Songyang Zhou, Tao Ni, Tao Yun, Tian Pei, Tian Ye, Tianyuan Yue, Wangding Zeng, Wen Liu, Wenfeng Liang, Wenjie Pang, Wenjing Luo, Wenjun Gao, Wentao Zhang, Xi Gao, Xiangwen Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaokang Zhang, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingkai Yu, Xingyou Li, Xinyu Yang, Xinyuan Li, Xu Chen, Xuecheng Su, Xuehai Pan, Xuheng Lin, Xuwei Fu, Y. Q. Wang, Yang Zhang, Yanhong Xu, Yanru Ma, Yao Li, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Qian, Yi Yu, Yichao Zhang, Yifan Ding, Yifan Shi, Yiliang Xiong, Ying He, Ying Zhou, Yinmin Zhong, Yishi Piao, Yisong Wang, Yixiao Chen, Yixuan Tan, Yixuan Wei, Yiyang Ma, Yiyuan Liu, Yonglun Yang, Yongqiang Guo, Yongtong Wu, Yu Wu, Yuan Cheng, Yuan Ou, Yuanfan Xu, Yudian Wang, Yue Gong, Yuhang Wu, Yuheng Zou, Yukun Li, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Z. F. Wu, Z. Z. Ren, Zehua Zhao, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhixian Huang, Zhiyu Wu, Zhuoshu Li, Zhuping Zhang, Zian Xu, Zihao Wang, Zihui Gu, Zijia Zhu, Zilin Li, Zipeng Zhang, Ziwei Xie, Ziyi Gao, Zizheng Pan, Zongqing Yao, Bei Feng, Hui Li, J. L. Cai, Jiaqi Ni, Lei Xu, Meng Li, Ning Tian, R. J. Chen, R. L. Jin, S. S. Li, Shuang Zhou, Tianyu Sun, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xinnan Song, Xinyi Zhou, Y. X. Zhu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu,

- Yunxian Ma, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, Dongjie Ji, Jian Liang, Jianzhong Guo, Jin Chen, Leyi Xia, Miaojun Wang, Mingming Li, Peng Zhang, Ruyi Chen, Shangmian Sun, Shaoqing Wu, Shengfeng Ye, T. Wang, W. L. Xiao, Wei An, Xianzu Wang, Xiaowen Sun, Xiaoxiang Wang, Ying Tang, Yukun Zha, Zekai Zhang, Zhe Ju, Zhen Zhang, and Zihua Qu. Deepseek-v3.2: Pushing the frontier of open large language models, 2025. URL <https://arxiv.org/abs/2512.02556>.
- Yichuan Deng, Zhao Song, and Tianyi Zhou. Superiority of softmax: Unveiling the performance edge over linear attention. *arXiv preprint arXiv:2310.11685*, 2023a.
- Yichuan Deng, Zhao Song, and Tianyi Zhou. Superiority of softmax: Unveiling the performance edge over linear attention. *arXiv preprint arXiv:2310.11685*, 2023b.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.
- Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. *International Conference on Learning Representations*, May 2021. URL <https://openreview.net/pdf?id=YicbFdNTTy>.
- Yao Gan, Yanyun Fu, Deyong Wang, and Yongming Li. A novel approach to attention mechanism using kernel functions: Kerformer. *Frontiers in Neurobotics*, 17, 08 2023. doi:10.3389/fnbot.2023.1214203.
- Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. The pile: An 800gb dataset of diverse text for language modeling. *CoRR*, abs/2101.00027, 2021. URL <https://arxiv.org/abs/2101.00027>.
- Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=tEYskw1VY2>.
- Albert Gu, Isys Johnson, Karan Goel, Khaled Saab, Tri Dao, Atri Rudra, and Christopher Ré. Combining recurrent, convolutional, and continuous-time models with linear state space layers. *Advances in neural information processing systems*, 34:572–585, 2021.
- Albert Gu, Karan Goel, and Christopher Re. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=uYLFoz1v1AC>.
- Anmol Gulati, James Qin, Chung-Cheng Chiu, Niki Parmar, Yu Zhang, Jiahui Yu, Wei Han, Shibo Wang, Zhengdong Zhang, Yonghui Wu, et al. Conformer: Convolution-augmented transformer for speech recognition. *arXiv preprint arXiv:2005.08100*, 2020.
- Ankit Gupta, Albert Gu, and Jonathan Berant. Diagonal state spaces are as effective as structured state spaces. *Advances in Neural Information Processing Systems*, 35:22982–22994, 2022.
- Dongchen Han, Yifan Pu, Zhuofan Xia, Yizeng Han, Xuran Pan, Xiu Li, Jiwen Lu, Shiji Song, and Gao Huang. Bridging the divide: Reconsidering softmax and linear attention. *Advances in Neural Information Processing Systems*, 37:79221–79245, 2024a.
- Dongchen Han, Yifan Pu, Zhuofan Xia, Yizeng Han, Xuran Pan, Xiu Li, Jiwen Lu, Shiji Song, and Gao Huang. Bridging the divide: Reconsidering softmax and linear attention. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024b. URL <https://openreview.net/forum?id=RSiGfzQap1>.
- Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, November 1997. ISSN 0899-7667. doi:10.1162/neco.1997.9.8.1735. Generated from Scopus record by KAUST IRTS on 2022-09-14.
- Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.
- John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Židek, Anna Potapenko, Alex Bridgland, Clemens Meyer, Simon A. A. Kohl, Andrew J. Ballard, Andrew Cowie, Bernardino Romera-Paredes, Stanislav Nikolov, Rishub Jain, Jonas Adler, Trevor Back, Stig Petersen, David Reiman, Ellen Clancy, Michal Zielinski, Martin Steinegger, Michalina Pacholska, Tamas Berghammer, Sebastian Bodenstern, David Silver, Oriol Vinyals, Andrew W. Senior, Koray Kavukcuoglu, Pushmeet Kohli, and Demis Hassabis. Highly accurate protein structure prediction with alphafold. *Nature*, 596(7873):583–589, Aug 2021. ISSN 1476-4687. doi:10.1038/s41586-021-03819-2. URL <https://doi.org/10.1038/s41586-021-03819-2>.

- R. E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 03 1960. ISSN 0021-9223. doi:10.1115/1.3662552. URL <https://doi.org/10.1115/1.3662552>.
- Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In Hal Daumé III and Aarti Singh, editors, *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 5156–5165. PMLR, 13–18 Jul 2020. URL <https://proceedings.mlr.press/v119/katharopoulos20a.html>.
- Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. *arXiv preprint arXiv:2001.04451*, 2020.
- Weijie Kong, Qi Tian, Zijian Zhang, Rox Min, Zuozhuo Dai, Jin Zhou, Jiangfeng Xiong, Xin Li, Bo Wu, Jianwei Zhang, Kathrina Wu, Qin Lin, Junkun Yuan, Yanxin Long, Aladdin Wang, Andong Wang, Changlin Li, DuoJun Huang, Fang Yang, Hao Tan, Hongmei Wang, Jacob Song, Jiawang Bai, Jianbing Wu, Jinbao Xue, Joey Wang, Kai Wang, Mengyang Liu, Pengyu Li, Shuai Li, Weiyan Wang, Wenqing Yu, Xincheng Deng, Yang Li, Yi Chen, Yutao Cui, Yuanbo Peng, Zhentao Yu, Zhiyu He, Zhiyong Xu, Zixiang Zhou, Zunnan Xu, Yangyu Tao, Qinglin Lu, Songtao Liu, Dax Zhou, Hongfa Wang, Yong Yang, Di Wang, Yuhong Liu, Jie Jiang, and Caesar Zhong. Hunyuanvideo: A systematic framework for large video generative models, 2025. URL <https://arxiv.org/abs/2412.03603>.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pages 611–626, 2023.
- Zhixuan Lin, Evgenii Nikishin, Xu He, and Aaron Courville. Forgetting transformer: Softmax attention with a forget gate. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=q2Lnyegkr8>.
- Hao Liu, Matei Zaharia, and Pieter Abbeel. Ring attention with blockwise transformers for near-infinite context. *arXiv preprint arXiv:2310.01889*, 2023.
- Chunjie Luo, Jianfeng Zhan, Xiaohe Xue, Lei Wang, Rui Ren, and Qiang Yang. Cosine normalization: Using cosine similarity instead of dot product in neural networks. In *Artificial Neural Networks and Machine Learning–ICANN 2018: 27th International Conference on Artificial Neural Networks, Rhodes, Greece, October 4-7, 2018, Proceedings, Part I* 27, pages 382–391. Springer, 2018.
- William Merrill, Yanhong Li, Tyler Romero, Anej Svete, Caia Costello, Pradeep Dasigi, Dirk Groeneveld, David Heineman, Bailey Kuehl, Nathan Lambert, Chuan Li, Kyle Lo, Saumya Malik, DJ Matusz, Benjamin Minixhofer, Jacob Morrison, Luca Soldaini, Finbarr Timbers, Pete Walsh, Noah A. Smith, Hannaneh Hajishirzi, and Ashish Sabharwal. Olmo hybrid: From theory to practice. *arXiv preprint arXiv:2602.19020*, 2026. URL <https://arxiv.org/abs/2602.19020>.
- Maxim Milakov and Natalia Gimelshein. Online normalizer calculation for softmax. *CoRR*, abs/1805.02867, 2018. URL <http://arxiv.org/abs/1805.02867>.
- Evan Miller. Attention is off by one, 2023. URL <https://www.evanmiller.org/attention-is-off-by-one.html>.
- MiniMax, Aonian Li, Bangwei Gong, Bo Yang, Boji Shan, Chang Liu, Cheng Zhu, Chunhao Zhang, Congchao Guo, Da Chen, Dong Li, Enwei Jiao, Gengxin Li, Guojun Zhang, Haohai Sun, Houze Dong, Jiada Zhu, Jiaqi Zhuang, Jiayuan Song, Jin Zhu, Jintao Han, Jingyang Li, Junbin Xie, Junhao Xu, Junjie Yan, Kaishun Zhang, Kecheng Xiao, Kexi Kang, Le Han, Leyang Wang, Lianfei Yu, Liheng Feng, Lin Zheng, Linbo Chai, Long Xing, Meizhi Ju, Mingyuan Chi, Mozhi Zhang, Peikai Huang, Pengcheng Niu, Pengfei Li, Pengyu Zhao, Qi Yang, Qidi Xu, Qiexiang Wang, Qin Wang, Qiuhui Li, Ruitao Leng, Shengmin Shi, Shuqi Yu, Sichen Li, Songquan Zhu, Tao Huang, Tianrun Liang, Weigao Sun, Weixuan Sun, Weiyu Cheng, Wenkai Li, Xiangjun Song, Xiao Su, Xiaodong Han, Xinjie Zhang, Xinzhu Hou, Xu Min, Xun Zou, Xuyang Shen, Yan Gong, Yingjie Zhu, Yipeng Zhou, Yiran Zhong, Yongyi Hu, Yuanxiang Fan, Yue Yu, Yufeng Yang, Yuhao Li, Yunan Huang, Yunji Li, Yunpeng Huang, Yunzhi Xu, Yuxin Mao, Zehan Li, Zekang Li, Zewei Tao, Zewen Ying, Zhaoyang Cong, Zhen Qin, Zhenhua Fan, Zhihang Yu, Zhuo Jiang, and Zijia Wu. Minimax-01: Scaling foundation models with lightning attention, 2025. URL <https://arxiv.org/abs/2501.08313>.
- Gabriel Mongaras and Eric C. Larson. On the expressiveness of softmax attention: A recurrent neural network perspective. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856. URL <https://openreview.net/forum?id=PHcIT0i3vV>.
- Gabriel Mongaras and Eric C Larson. 2mamba2furious: Linear in complexity, competitive in accuracy. *arXiv preprint arXiv:2602.17363*, 2026.
- Gabriel Mongaras, Trevor Dohm, and Eric C Larson. Cottention: Linear transformers with cosine attention. *Proceedings of the 2025 Computing Conference*, 2025.

- Tobias Christian Nauen, Sebastian Palacio, and Andreas Dengel. Taylorshift: Shifting the complexity of self-attention from squared to linear (and back) using taylor-softmax. In *ICPR (6)*, pages 1–16, 2024. URL https://doi.org/10.1007/978-3-031-78172-8_1.
- Huy Q Nguyen, Cuong Q Nguyen, Dung D Le, and Hieu H Pham. Enhancing few-shot image classification with cosine transformer. *IEEE Access*, 2023.
- NVIDIA Corporation. Nvidia a100 tensor core gpu architecture. Technical report, NVIDIA Corporation, May 2020. URL <https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf>. v1.0.
- William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 4195–4205, 2023.
- Guilherme Penedo, Hynek Kydlíček, Loubna Ben allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. The fineweb datasets: Decanting the web for the finest text data at scale. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2024. URL <https://openreview.net/forum?id=n6SCkn2QaG>.
- Bo Peng, Ruichong Zhang, Daniel Goldstein, Eric Alcaide, Haowen Hou, Janna Lu, William Merrill, Guangyu Song, Kaifeng Tan, Saiteja Utpala, et al. Rwkv-7" goose" with expressive dynamic state evolution. *arXiv preprint arXiv:2503.14456*, 2025.
- Hao Peng, Nikolaos Pappas, Dani Yogatama, Roy Schwartz, Noah A. Smith, and Lingpeng Kong. Random feature attention. In *9th International Conference on Learning Representations, ICLR 2021*, 2021.
- Zhen Qin, Weixuan Sun, Hui Deng, Dongxu Li, Yunshen Wei, Baohong Lv, Junjie Yan, Lingpeng Kong, and Yiran Zhong. cosformer: Rethinking softmax in attention. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=B18CQrx2Up4>.
- Markus N Rabe and Charles Staats. Self-attention does not need $o(n^2)$ memory. *arXiv preprint arXiv:2112.05682*, 2021.
- Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. Improving language understanding by generative pre-training. *OpenAI Blog*, 2018.
- Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PmlR, 2021.
- Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine Mcleavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett, editors, *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 28492–28518. PMLR, 23–29 Jul 2023. URL <https://proceedings.mlr.press/v202/radford23a.html>.
- Aditya Ramesh, Prafulla Dhariwal, Alex Nichol, Casey Chu, and Mark Chen. Hierarchical text-conditional image generation with clip latents, 2022. URL <https://arxiv.org/abs/2204.06125>.
- David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986. doi:10.1038/323533a0. URL <https://doi.org/10.1038/323533a0>.
- Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. Flashattention-3: Fast and accurate attention with asynchrony and low-precision. *Advances in Neural Information Processing Systems*, 37: 68658–68685, 2024.
- Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.
- Noam Shazeer. GLU variants improve transformer. *CoRR*, abs/2002.05202, 2020. URL <https://arxiv.org/abs/2002.05202>.
- Noam Shazeer, *Azalia Mirhoseini, *Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017. URL <https://openreview.net/forum?id=B1ckMDqlg>.
- Zhiqiang Shen, Tianhua Tao, Liqun Ma, Willie Neiswanger, Zhengzhong Liu, Hongyi Wang, Bowen Tan, Joel Hestness, Natalia Vassilieva, Daria Soboleva, and Eric Xing. SlimPajama-DC: Understanding Data Combinations for LLM Training. *arXiv e-prints*, art. arXiv:2309.10818, September 2023. doi:10.48550/arXiv.2309.10818.
- Jerome Sieber, Carmen Amo Alonso, Alexandre Didier, Melanie Zeilinger, and Antonio Orvieto. Understanding the differences in foundation models: Attention, state space models, and recurrent neural networks. In *The Thirty-eighth*

- Annual Conference on Neural Information Processing Systems*, 2024. URL <https://openreview.net/forum?id=iF7MnXnxRw>.
- Ethan Smith. Softmax attention is a fluke, 2025. URL <https://www.ethansmith2000.com/post/softmax-attention-is-a-fluke>.
- Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024. ISSN 0925-2312. doi:<https://doi.org/10.1016/j.neucom.2023.127063>. URL <https://www.sciencedirect.com/science/article/pii/S0925231223011864>.
- Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, Tatsunori Hashimoto, and Carlos Guestrin. Learning to (learn at test time): RNNs with expressive hidden states, 2025. URL <https://openreview.net/forum?id=eifW0W0xgt>.
- Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models, 2023. URL <https://arxiv.org/abs/2307.08621>.
- Yi Tay, Mostafa Dehghani, Dara Bahri, and Donald Metzler. Efficient transformers: A survey. *ACM Computing Surveys*, 55(6):1–28, 2022.
- Kimi Team, Yu Zhang, Zongyu Lin, Xingcheng Yao, Jiaxi Hu, Fanqing Meng, Chengyin Liu, Xin Men, Songlin Yang, Zhiyuan Li, Wentao Li, Enzhe Lu, Weizhou Liu, Yanru Chen, Weixin Xu, Longhui Yu, Yejie Wang, Yu Fan, Longguang Zhong, Enming Yuan, Dehao Zhang, Yizhi Zhang, T. Y. Liu, Haiming Wang, Shengjun Fang, Weiran He, Shaowei Liu, Yiwei Li, Jianlin Su, Jiezhong Qiu, Bo Pang, Junjie Yan, Zhejun Jiang, Weixiao Huang, Bohong Yin, Jiacheng You, Chu Wei, Zhengtao Wang, Chao Hong, Yutian Chen, Guanduo Chen, Yucheng Wang, Huabin Zheng, Feng Wang, Yibo Liu, Mengnan Dong, Zheng Zhang, Siyuan Pan, Wenhao Wu, Yuhao Wu, Longyu Guan, Jiawen Tao, Guohong Fu, Xinran Xu, Yuzhi Wang, Guokun Lai, Yuxin Wu, Xinyu Zhou, Zhilin Yang, and Yulun Du. Kimi linear: An expressive, efficient attention architecture, 2025. URL <https://arxiv.org/abs/2510.26692>.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. LLaMA: Open and Efficient Foundation Language Models. *arXiv e-prints*, art. arXiv:2302.13971, February 2023. doi:10.48550/arXiv.2302.13971.
- Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models, 2023. URL <https://arxiv.org/abs/2307.09288>.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.
- Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations*, 2018. URL <https://openreview.net/forum?id=rJXMpikCZ>.
- Dustin Wang, Rui-Jie Zhu, Steven Abreu, Yong Shan, Taylor Kergan, Yuqi Pan, Yuhong Chou, Zheng Li, Ge Zhang, Wenhao Huang, and Jason Eshraghian. A systematic analysis of hybrid linear attention, 2025. URL <https://arxiv.org/abs/2507.06457>.
- Maurice Weber, Daniel Fu, Quentin Anthony, Yonatan Oren, Shane Adams, Anton Alexandrov, Xiaozhong Lyu, Huu Nguyen, Xiaozhe Yao, Virginia Adams, Ben Athiwaratkun, Rahul Chalamala, Kezhen Chen, Max Ryabinin, Tri Dao, Percy Liang, Christopher Ré, Irina Rish, and Ce Zhang. Redpajama: an open dataset for training large language models, 2024. URL <https://arxiv.org/abs/2411.12372>.

- Bernard Widrow and Marcian E. Hoff. Adaptive switching circuits. In *1960 IRE WESCON Convention Record, Part 4*, pages 96–104, New York, 8 1960. Institute of Radio Engineers, Institute of Radio Engineers. URL <http://www-isl.stanford.edu/~widrow/papers/c1960adaptiveswitching.pdf>.
- Enze Xie, Junsong Chen, Junyu Chen, Han Cai, Haotian Tang, Yujun Lin, Zhekai Zhang, Muyang Li, Ligeng Zhu, Yao Lu, and Song Han. SANA: Efficient high-resolution text-to-image synthesis with linear diffusion transformers. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=N80j1XhtYZ>.
- Yunyang Xiong, Zhanpeng Zeng, Rudrasis Chakraborty, Mingxing Tan, Glenn Fung, Yin Li, and Vikas Singh. Nyströmformer: A nyström-based algorithm for approximating self-attention, 2021. URL <https://arxiv.org/abs/2102.03902>.
- Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. Gated linear attention transformers with hardware-efficient training. In Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp, editors, *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 56501–56523. PMLR, 21–27 Jul 2024a. URL <https://proceedings.mlr.press/v235/yang24ab.html>.
- Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024b. URL <https://openreview.net/forum?id=y8Rm4VNRPH>.
- Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=r8H7xhYPwz>.
- Ted Zadouri, Markus Hoehnerbach, Jay Shah, Timmy Liu, Vijay Thakkar, and Tri Dao. Flashattention-4: Algorithm and kernel pipelining co-design for asymmetric hardware scaling. *arXiv preprint arXiv:2603.05451*, 2026.
- Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, et al. Big bird: Transformers for longer sequences. *Advances in Neural Information Processing Systems*, 33, 2020.
- Michael Zhang, Kush Bhatia, Hermann Kumbong, and Christopher Re. The hedgehog & the porcupine: Expressive linear attentions with softmax mimicry. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=4g0212N2Nx>.
- Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pages 11106–11115, 2021.

A Softmax is an RNN - Model Ablation Parameters

Unless otherwise mentioned, the below are the parameters we used in our models. As our base model is llama 2 (Touvron et al., 2023). RoPE (Su et al., 2024) is used on the attention matrix and the MLPs follow SwiGLU (Shazeer, 2020).

1. batch size - 36
2. learning rate - 1e-4
3. warmup steps - 10,000
4. warmup type - linear warmup from 0, linear decay
5. num steps - 100,000
6. precision - float32 and bfloat16 mixed precision
7. Weight decay - 0.01
8. Max sequence length - 1024 for general experiments, 4096 for length scaling experiment
9. Test percentage - 0.001
10. Optimizer - AdamW
11. Adam betas - 0.9 and 0.999
12. Hidden size - 1024 (3072 for the large model)

13. MLP intermediate size - 2048 (6144 for the large model)
14. Num attention heads - 16
15. Num hidden layers - 20
16. Tokenizer - llama2-7b-hf
17. Gradient clipping - 1.0 clipping for gated models, no clipping for all other experiments

Each model was trained for a maximum of 2 days. For most experiments, we use distributed data parallel processing to train on two 80 GB, A100 GPUs with the exception of the large model, trained on 4 GPUs, and 4096 sequence length, trained on 6 GPUs.

B Expanded Results Figures

Figure 23 shows expanded plots for the comparison of our recurrent softmax attention formulation.

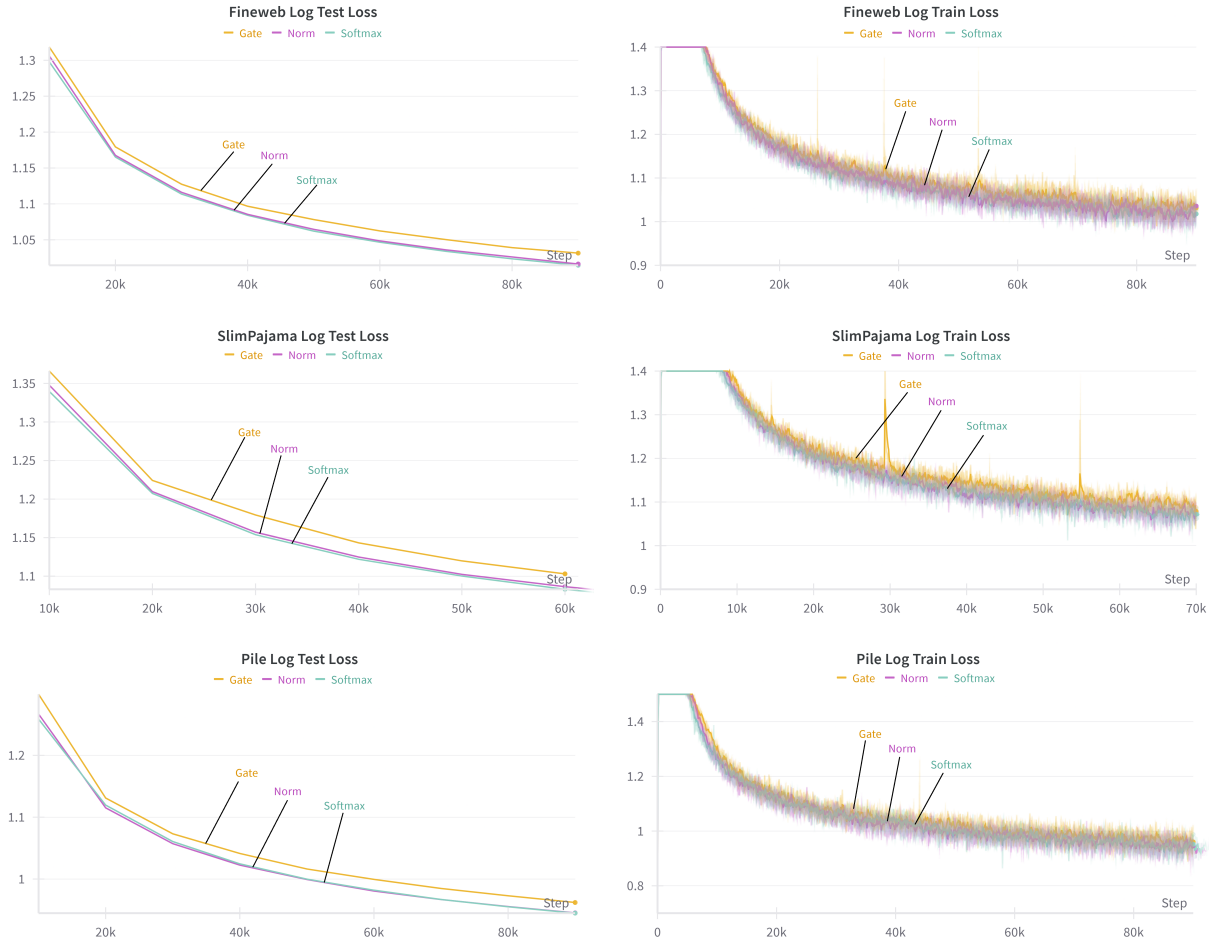


Figure 23: Expanded test and train loss plots on various datasets for softmax attention and the proposed methods with gate or norm replacements.

In Figure 24, we show expanded plots of our ablation study.

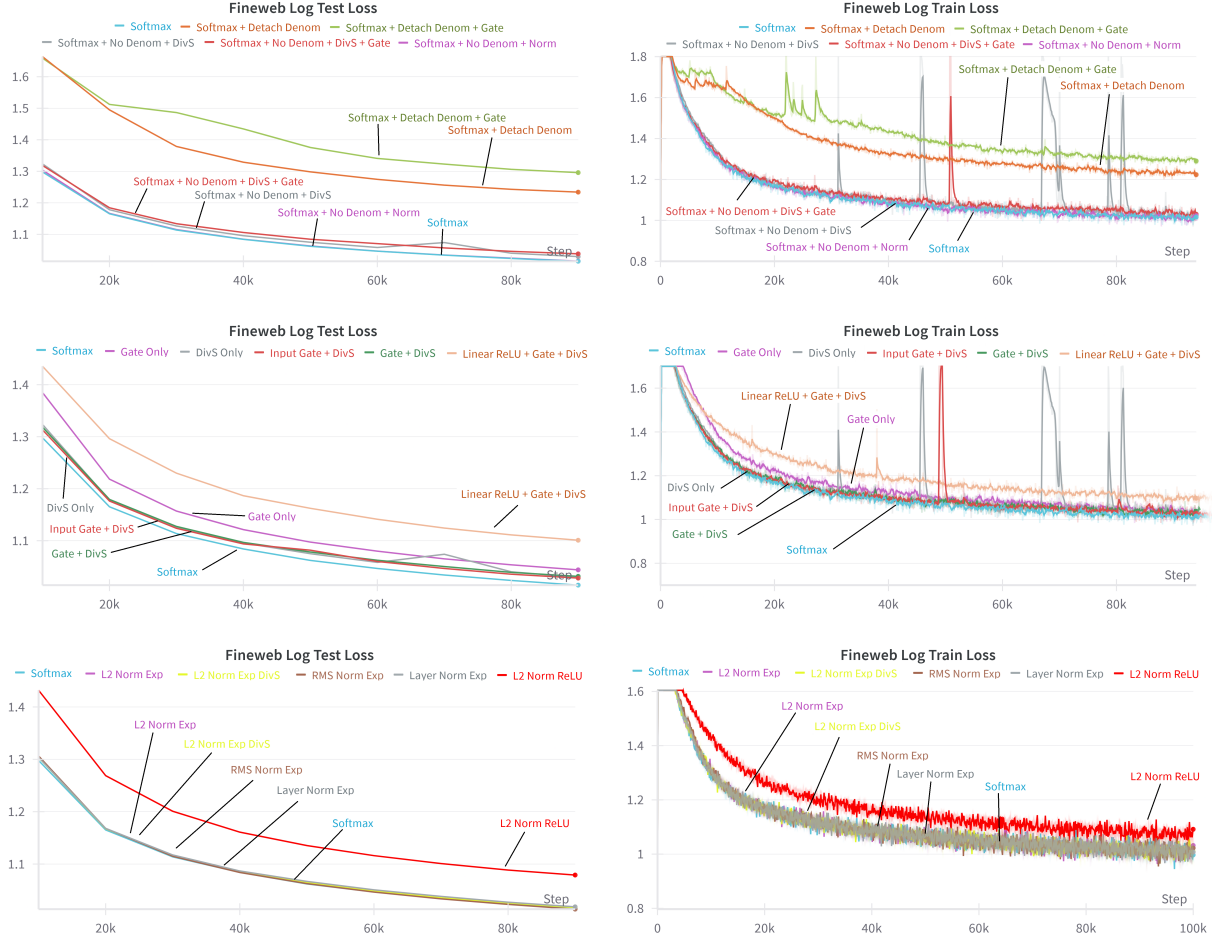


Figure 24: Expanded test and train loss on various datasets for softmax attention and the proposed methods with gate or norm replacements.

C 2Mamba2Furious - Model Ablation Details

In this section, we briefly mention the setup for all ablations. Specifically, the llama 2 (Touvron et al., 2023) architecture is used as our base model. To ablate the changes to the attention mechanism, we replace the attention mechanism in all layers of the llama 2 model with the ablated variations, keeping all other parts of the architecture constant. The base max sequence length is 2048, however 4096 and 8192 sequence lengths are used for testing models on longer context lengths. Most experiments are done with a base model of about 300 million parameters. To test scale we increase the size to 700 million parameters. The following is information regarding our training setup:

- dataset: HuggingFaceFW/fineweb (CC-MAIN-2024-51 version)
- test dataset: 0.1 percent of the train data (seed 123)
- model name: meta-llama/Llama-2-7b-hf
- optimizer: AdamW (0.9, 0.999 betas)
- batch size (over all devices): 32
- learning rate: 1e-4
- warmup steps: 10,000
- total steps: 100,000
- weight decay: 0.01

- no grad clipping

The 300 million parameter model (small) has the following setup:

- hidden size: 1024
- intermediate MLP size: 2048
- hidden MLP activation: silu
- num attention heads: 16 (64 head dim)
- num layers: 20
- vocab size: 32,000 (llama 2 vocab)

The 700 million parameter model (medium) has the following setup:

- hidden size: 1536
- intermediate MLP size: 3072
- hidden MLP activation: silu
- num attention heads: 24 (64 head dim)
- num layers: 27
- vocab size: 32,000 (llama 2 vocab)

D 2Mamba2Furious Gradients

We derive the gradient equations for all necessary kernels and provide them in this section. Each gradient assumes the input is the query, key, and value matrices and the output is the post-attention outputs. Each kernel performs the attention operation, but no pre-processing or post-processing.

D.1 Linear

$$\begin{aligned}
 O &= (QK^T \odot M) V \\
 \frac{\partial L}{\partial Q} &= \left(\frac{\partial L}{\partial O} V^T \odot M \right) K \\
 \frac{\partial L}{\partial K} &= \left(V \frac{\partial L}{\partial O}^T \odot M^T \right) Q \\
 \frac{\partial L}{\partial V} &= (QK^T \odot M)^T \frac{\partial L}{\partial O}
 \end{aligned}$$

D.2 Linear with Softmax Norm

$$\begin{aligned}
 O &= \frac{QK^T \odot M}{\sum (QK^T \odot M)} V \\
 Y &= QK^T \odot M \\
 S &= \sum_j Y \\
 Y_N &= \text{Norm}(Y) = \frac{Y}{N} = \frac{Y}{\sum Y} \\
 G &= \frac{\partial L}{\partial O} V^T \odot M \\
 D &= \frac{G - \sum_j (Y_N \odot G)}{S} \odot M \\
 \frac{\partial L}{\partial Q} &= DK \\
 \frac{\partial L}{\partial K} &= D^T Q \\
 \frac{\partial L}{\partial V} &= Y_N^T \frac{\partial L}{\partial O}
 \end{aligned}$$

D.3 Linear with A-Gate

Note: Typically we use M to denote the causal mask. Here, we additionally use M to denote the row (key and value) axis whereas the column (query) axis is denoted by N . Additionally, $\sum_j$ denotes a summation along the M axis while $\sum_i$ denotes a summation along the N axis. The differentiation is necessary for the gradients of the A -mask. In the self-attention case, $N = M$, however we still make this differentiation for the gradient derivation below.

$$\begin{aligned}
 A_M &= e^{A-A^T} \in \mathbb{R}^{B,H,N,M}, \quad A \in \mathbb{R}^{B,H,N} \\
 O &= (QK^T \odot M \odot A_M) V \\
 \frac{\partial L}{\partial Q} &= \left(\frac{\partial L}{\partial O} V^T \odot M \odot A_M \right) K \\
 \frac{\partial L}{\partial K} &= \left(\frac{\partial L}{\partial O} V^T \odot M \odot A_M \right)^T Q \\
 \frac{\partial L}{\partial V} &= (QK^T \odot M \odot A_M)^T \frac{\partial L}{\partial O} \\
 \frac{\partial L}{\partial A_M^N} &= \sum_j \left(QK^T \odot M \odot A_M \odot \frac{\partial L}{\partial O} V^T \right) \\
 \frac{\partial L}{\partial A_M^M} &= - \sum_i \left(QK^T \odot M \odot A_M \odot \frac{\partial L}{\partial O} V^T \right) \\
 \frac{\partial L}{\partial A_M} &= \frac{\partial L}{\partial A_M^N} + \frac{\partial L}{\partial A_M^M}
 \end{aligned}$$

D.4 Squared with A-mask

$$\begin{aligned}
 A_M &= e^{A-A^T} \in \mathbb{R}^{B,H,N,M}, \quad A \in \mathbb{R}^{B,H,N} \\
 O &= ((QK^T)^2 \odot M \odot A_M) V \\
 \frac{\partial L}{\partial Q} &= \left(2 \odot QK^T \odot \frac{\partial L}{\partial O} V^T \odot M \odot A_M \right) K \\
 \frac{\partial L}{\partial K} &= \left(2 \odot QK^T \odot \frac{\partial L}{\partial O} V^T \odot M \odot A_M \right)^T Q \\
 \frac{\partial L}{\partial V} &= ((QK^T)^2 \odot M \odot A_M)^T \frac{\partial L}{\partial O} \\
 \frac{\partial L}{\partial A_M^N} &= \sum_j \left((QK^T)^2 \odot M \odot A_M \odot \frac{\partial L}{\partial O} V^T \right) \\
 \frac{\partial L}{\partial A_M^M} &= - \sum_i \left((QK^T)^2 \odot M \odot A_M \odot \frac{\partial L}{\partial O} V^T \right) \\
 \frac{\partial L}{\partial A_M} &= \frac{\partial L}{\partial A_M^N} + \frac{\partial L}{\partial A_M^M}
 \end{aligned}$$

D.5 Squared with A-mask and Softmax Norm (2Mamba)

$$\begin{aligned}
 O &= \frac{(QK^T)^2 \odot M \odot A_M}{\sum \left((QK^T)^2 \odot M \odot A_M \right)} V & \frac{\partial L}{\partial Q} &= DK \\
 Y &= (QK^T)^2 \odot M \odot A_M & \frac{\partial L}{\partial K} &= D^T Q \\
 S &= \sum_j Y & \frac{\partial L}{\partial V} &= Y_N^T \frac{\partial L}{\partial O} \\
 Y_N &= \text{Norm}(Y) = \frac{Y}{N} = \frac{Y}{\sum Y} & \frac{\partial L}{\partial A_M^N} &= \sum_j \left((QK^T)^2 \odot M \odot A_M \odot G \right) \\
 G &= \frac{\partial L}{\partial O} V^T \odot M - \sum_j \left(Y_N \odot \frac{\partial L}{\partial O} V^T \right) & \frac{\partial L}{\partial A_M^M} &= - \sum_i \left((QK^T)^2 \odot M \odot A_M \odot G \right) \\
 D &= \frac{2 \odot QK^T \odot G}{S} \odot M & \frac{\partial L}{\partial A_M} &= \frac{\partial L}{\partial A_M^N} + \frac{\partial L}{\partial A_M^M}
 \end{aligned}$$

D.6 Exponentiated with A-mask and Softmax Norm (2Mamba-E)

$$\begin{aligned}
 O &= \frac{e^{QK^T} \odot M \odot A_M}{\sum \left(e^{QK^T} \odot M \odot A_M \right)} V & \frac{\partial L}{\partial Q} &= DK^T \\
 Y &= e^{QK^T} \odot M \odot A_M & \frac{\partial L}{\partial K} &= D^T Q \\
 S &= \sum_j Y & \frac{\partial L}{\partial V} &= Y_N^T \frac{\partial L}{\partial O} \\
 Y_N &= \text{Norm}(Y) = \frac{Y}{N} = \frac{Y}{\sum Y} & \frac{\partial L}{\partial A_M^N} &= \sum_j D \\
 G &= \frac{\partial L}{\partial O} V^T \odot M - \sum_j \left(O \odot \frac{\partial L}{\partial O} \right) & \frac{\partial L}{\partial A_M^M} &= - \sum_i D \\
 D &= Y_N \odot G \odot M & \frac{\partial L}{\partial A_M} &= \frac{\partial L}{\partial A_M^N} + \frac{\partial L}{\partial A_M^M}
 \end{aligned}$$